VOL. 10

SQL

Ask questions of your data with PostgreSQL

Source	w3schools.com/sql
Generated	2026-10-10
Level	Beginner
Estimated study time	20 hours
Language	English
Volume	Vol. 10 - SQL

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 6

Chapter map 6

Chapter 1 - Databases, tables, and the sample schema 7

1.1 Summary 7

1.2 Concepts 7

A database you talk to 7

Tables, rows and columns 8

Keys and constraints 8

The sample schema 8

PostgreSQL choices worth knowing 9

1.3 Examples 9

1.4 Common mistakes and best practices 12

1.5 Practice exercises 13

Solutions 13

1.6 Mini test 14

Answer key and explanations 14

Chapter 2 - SQL syntax and your first queries 14

2.1 Summary 14

2.2 Concepts 15

The shape of a query 15

Statements, semicolons and case 15

Identifiers and quoting 15

Comments 16

2.3 Examples 16

2.4 Common mistakes and best practices 18

2.5 Practice exercises 19

Solutions 19

2.6 Mini test 19

Answer key and explanations 20

Chapter 3 - Choosing columns: aliases and DISTINCT 20

3.1 Summary 20

3.2 Concepts 20

Projection: choosing what you see . . . 21
Column aliases . . . 21
Table aliases . . . 21
DISTINCT . . . 21
3.3 Examples . . . 21
3.4 Common mistakes and best practices . . . 24
3.5 Practice exercises . . . 25
Solutions . . . 25
3.6 Mini test . . . 25
Answer key and explanations . . . 26
Chapter 4 - Filtering rows with WHERE . . . 26
4.1 Summary . . . 26
4.2 Concepts . . . 26
The filter in the pipeline . . . 26
Comparison operators . . . 27
Boolean logic . . . 27
IN and BETWEEN . . . 27
Pattern matching with LIKE . . . 28
NULL is not a value . . . 28
4.3 Examples . . . 28
4.4 Common mistakes and best practices . . . 31
4.5 Practice exercises . . . 32
Solutions . . . 32
4.6 Mini test . . . 32
Answer key and explanations . . . 33
Chapter 5 - Sorting and limiting results . . . 33
5.1 Summary . . . 33
5.2 Concepts . . . 33
ORDER BY and its keys . . . 33
Where NULL goes . . . 34
LIMIT and OFFSET . . . 34
5.3 Examples . . . 34
5.4 Common mistakes and best practices . . . 37
5.5 Practice exercises . . . 38
Solutions . . . 38

5.6 Mini test . . . 38
Answer key and explanations . . . 38
Chapter 6 - Changing data: INSERT, UPDATE and DELETE . . . 39
6.1 Summary . . . 39
6.2 Concepts . . . 39
The three data-changing statements . . . 39
Transactions . . . 39
Copying rows between tables . . . 40
Keep values out of the SQL text . . . 40
6.3 Examples . . . 40
6.4 Common mistakes and best practices . . . 44
6.5 Practice exercises . . . 44
Solutions . . . 44
6.6 Mini test . . . 45
Answer key and explanations . . . 46
Chapter 7 - NULL, dates and conditional expressions . . . 46
7.1 Summary . . . 46
7.2 Concepts . . . 46
Three-valued logic . . . 46
The NULL-handling functions . . . 47
Dates, times and intervals . . . 47
CASE expressions . . . 47
7.3 Examples . . . 48
7.4 Common mistakes and best practices . . . 51
7.5 Practice exercises . . . 51
Solutions . . . 52
7.6 Mini test . . . 52
Answer key and explanations . . . 52
Chapter 8 - Joins . . . 52
8.1 Summary . . . 53
8.2 Concepts . . . 53
What a join does . . . 53
The join types . . . 53
ON and WHERE in an outer join . . . 53
A table can join itself . . . 54

8.3 Examples . . . 54
8.4 Common mistakes and best practices . . . 57
8.5 Practice exercises . . . 57
Solutions . . . 57
8.6 Mini test . . . 58
Answer key and explanations . . . 58
Chapter 9 - Aggregating, grouping and combining results . . . 58
9.1 Summary . . . 59
9.2 Concepts . . . 59
The aggregate functions . . . 59
The grouping pipeline . . . 59
Set operations . . . 60
Subqueries . . . 60
Views . . . 60
9.3 Examples . . . 60
9.4 Common mistakes and best practices . . . 67
9.5 Practice exercises . . . 67
Solutions . . . 67
9.6 Mini test . . . 68
Answer key and explanations . . . 68
Cheat sheet . . . 68
10.1 A query in order . . . 68
10.2 Operators and predicates . . . 69
10.3 Joins . . . 69
10.4 Aggregation and set operations . . . 69
10.5 NULL, dates and CASE . . . 70
10.6 Changing data . . . 70
10.7 DDL quick reference . . . 71
10.8 Safe queries checklist . . . 71
Final test . . . 71
11.1 Final test answers . . . 72
Sources . . . 73

How to use this volume

Prerequisites: none. No programming experience is assumed. If you can edit a text file and run a command in a terminal, you can do every example here.

The dialect. Every statement in this volume is normalised to PostgreSQL, the free, open-source relational database server. The material behind this book teaches MySQL, SQL Server and MS Access side by side, so the same idea often appears three times in three dialects; here you learn one consistent form, and the few places where other databases really differ (for example `LIMIT` versus `TOP`) are pointed out instead of repeated.

Estimated study time: about 20 hours, exercises and tests included. SQL rewards practice more than reading, so plan on typing every statement yourself.

One schema for the whole volume. Chapter 1 creates a small library database with four tables and fills it with sample rows. Every later example queries that same data, so each result table can be reproduced exactly. The expected outputs were traced through that sample data; if your numbers differ, your rows differ.

How the chapters work: each chapter opens with a summary, explains the concepts, shows numbered examples with their expected result, lists the common mistakes with best practices, and closes with practice exercises, their solutions, and a mini test with an answer key. All answers are included.

How to run the code

shell

```shell
# 1. create the database once (skip if it already exists)
createdb -U postgres learn_sql

# 2. load the schema and the sample rows from Chapter 1 into it
psql -U postgres -d learn_sql -f 10-sql-schema.sql

# 3. open an interactive prompt and paste any statement from this book
psql -U postgres -d learn_sql
```

psql ships with the PostgreSQL installer on Windows, macOS and Linux. Inside the prompt, type a statement, end it with a semicolon, and press Enter; the result prints immediately. `\q` leaves the prompt, `\dt` lists your tables.

Chapter map

#	Chapter	You will learn
1	Databases, tables, and the sample schema	tables, keys, constraints, the library database used all volume
2	SQL syntax and your first queries	statements, comments, column lists, expressions
3	Choosing columns: aliases and DISTINCT	projection, renaming columns, removing duplicates

#	Chapter	You will learn
4	Filtering rows with WHERE	comparisons, AND/OR/NOT, IN, BETWEEN, LIKE, IS NULL
5	Sorting and limiting results	ORDER BY, LIMIT, OFFSET, NULL ordering
6	Changing data: INSERT, UPDATE, DELETE	adding, changing and removing rows, transactions, safe queries
7	NULL, dates and conditional expressions	three-valued logic, COALESCE, NULLIF, CASE, date functions
8	Joins	INNER, LEFT, RIGHT, FULL and SELF joins
9	Aggregating, grouping and combining results	aggregates, GROUP BY, HAVING, UNION, subqueries, EXISTS
10	Cheat sheet	compact reference tables
11	Final test	25 questions with answers

Chapter 1 - Databases, tables, and the sample schema

1.1 Summary

- SQL is the language used to create, query and change the data held in a relational database.
- Data is stored in tables, and a table is a grid of rows (records) and columns (fields).
- A primary key identifies every row of its table uniquely and is never NULL.
- A foreign key points from one table to the primary key of another and keeps the two tables consistent.
- Constraints such as NOT NULL, UNIQUE, CHECK and DEFAULT reject wrong data before it is stored.
- PostgreSQL is a free, open-source relational server, and psql is the command-line client that sends it SQL.
- This volume uses one small library schema, so every example runs against exactly the same rows.
- DDL statements such as CREATE TABLE describe the structure, while DML statements such as INSERT and SELECT work on the rows.
- Tables are linked by keys instead of being nested, which is what makes joins possible later.

1.2 Concepts

A database you talk to

```
you: psql, a GUI tool, or an application
     |
     |  SELECT title FROM books;
     v
+-----------------------------------------------------+
| PostgreSQL server                                   |
|  1. parses the statement                            |
|  2. plans how to read the tables                    |
|  3. executes it and builds a result set of rows     |
+-----------------------------------------------------+
     |
     |  result set: rows with named columns
     v
printed back on your screen
```

The server owns the data. Your client only sends text and prints what comes back, so any tool that can send SQL can read the same database.

Tables, rows and columns

A table is one entity you want to remember: authors, books, members, loans. Each row is one thing of that kind, and each column is one fact about it. A cell holds a single value of a single type: text, a whole number, a decimal amount, a date, or NULL, which means "no value recorded".

Keys and constraints

Constraint	Guarantees	Where it appears in this volume
`NOT NULL`	the cell always has a value	`books.title`
`UNIQUE`	no two rows share the value	`members.email`
`PRIMARY KEY`	unique, and never NULL; identifies the row	`books.book_id`
`FOREIGN KEY`	the value must exist in the referenced table	`books.author_id`
`CHECK`	the value must satisfy a rule	`books.price >= 0`
`DEFAULT`	value used when the INSERT supplies none	none in this schema

Together these rules are the schema's guard rails: they turn many kinds of bad data into a clear error message at write time instead of silent corruption.

The sample schema

```
authors 1 ─────< N books      one author can write many books
members 1 ─────< N loans      one member can borrow many times
books   1 ─────< N loans      one book can be borrowed many times
```

The four tables in full:

```
authors                                 books
  author_id      primary key              book_id          primary key
  first_name     not null                 title            not null
  last_name      not null                 author_id        foreign key -> authors
  birth_date     date, may be NULL        genre            text
  country        text                     price            decimal, >= 0
                                          pages            whole number, > 0
members                                    published_year   whole number
  member_id      primary key
  full_name      not null                 loans
  email          unique                     loan_id          primary key
  joined_on      not null                   book_id          foreign key -> books
  city           text, may be NULL          member_id        foreign key -> members
                                            loan_date        not null
                                            return_date      NULL while the book is out
```

Two columns are deliberately allowed to be NULL, because real data has holes: Frank Herbert's birth date is unknown in this data set, and Radia Perlman's record has no city. Chapter 7 works through what NULL does to your queries.

PostgreSQL choices worth knowing

- `GENERATED ALWAYS AS IDENTITY` (PostgreSQL 10 and later) fills primary keys automatically; the older spelling was `SERIAL`.
- Money belongs in `NUMERIC`, whole counts in `INTEGER`, calendar days in `DATE`, and short text in `VARCHAR(n)`.
- PostgreSQL limits rows with `LIMIT`; SQL Server spells the same idea `TOP`, and MS Access has no direct equivalent.

1.3 Examples

Every statement in this chapter is the setup for the whole volume. Run the chapter once, from top to bottom, and every query in Chapters 2 to 9 will return the tables shown there.

Example 1: create the `authors` table.

SQL

```sql
CREATE TABLE authors (
    author_id  INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name  VARCHAR(50) NOT NULL,
    birth_date DATE,
    country    VARCHAR(50)
);
```

output

```
CREATE TABLE
```

The identity column assigns `1, 2, 3, ...` to each inserted row, so you never type an id by hand. `birth_date` has no `NOT NULL`, so a row with an unknown birth date is still accepted.

Example 2: create the `books` table with a foreign key and a check.

SQL

```sql
CREATE TABLE books (
    book_id        INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    title          VARCHAR(120) NOT NULL,
    author_id      INTEGER NOT NULL REFERENCES authors (author_id),
    genre          VARCHAR(40),
    price          NUMERIC(6,2) CHECK (price >= 0),
    pages          INTEGER CHECK (pages > 0),
    published_year SMALLINT
);
```

output

```
CREATE TABLE
```

`author_id` references `authors (author_id)`, so a book row can only name an author that exists. `NUMERIC(6,2)` stores amounts exactly, which is why it is the right type for money, and the `CHECK` refuses negative prices.

Example 3: create the `members` table.

SQL

```sql
CREATE TABLE members (
    member_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    full_name VARCHAR(80) NOT NULL,
    email     VARCHAR(120) NOT NULL UNIQUE,
    joined_on DATE NOT NULL,
    city      VARCHAR(60)
);
```

output

```
CREATE TABLE
```

`UNIQUE` on `email` means two members cannot share an address; `city` may stay empty.

Example 4: create the `loans` table, which points at both other tables.

SQL

```sql
CREATE TABLE loans (
    loan_id     INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    book_id     INTEGER NOT NULL REFERENCES books (book_id),
    member_id   INTEGER NOT NULL REFERENCES members (member_id),
    loan_date   DATE NOT NULL,
    return_date DATE
);
```

output

```
CREATE TABLE
```

A loan record is meaningless without a book and a member, so both foreign keys are `NOT NULL`. `return_date` is NULL until the book comes back.

Example 5: insert the authors. Note that the id column is not mentioned; the identity column fills it.

SQL

```sql
INSERT INTO authors (first_name, last_name, birth_date, country) VALUES
    ('Ursula',     'Le Guin',        DATE '1929-10-21', 'United States'),
    ('Gabriel',    'Garcia Marquez', DATE '1927-03-06', 'Colombia'),
    ('Jane',       'Austen',         DATE '1775-12-16', 'United Kingdom'),
    ('Haruki',     'Murakami',       DATE '1949-01-12', 'Japan'),
    ('Chimamanda', 'Ngozi Adichie',  DATE '1977-09-15', 'Nigeria'),
    ('Frank',      'Herbert',        NULL,              'United States'),
    ('Kazuo',      'Ishiguro',       DATE '1954-11-08', 'Japan');
```

output

```
INSERT 0 7
```

`INSERT 0 7` reads as "0 rows skipped, 7 rows inserted". The `DATE '...'` literal is written in ISO form, year first, which PostgreSQL never misreads. Author 6 keeps a NULL birth date, and author 7 has no books at all.

Example 6: insert the books.

SQL

```sql
INSERT INTO books (title, author_id, genre, price, pages, published_year) VALUES
    ('A Wizard of Earthsea',           1, 'Fantasy',           12.99, 183, 1968),
    ('The Left Hand of Darkness',      1, 'Science fiction',   14.50, 304, 1969),
    ('One Hundred Years of Solitude',  2, 'Magical realism',   15.20, 417, 1967),
    ('Love in the Time of Cholera',    2, 'Romance',           13.75, 348, 1985),
    ('Pride and Prejudice',            3, 'Romance',            9.99, 279, 1813),
    ('Norwegian Wood',                 4, 'Fiction',           11.40, 296, 1987),
    ('Kafka on the Shore',             4, 'Fiction',           13.00, 505, 2002),
    ('Half of a Yellow Sun',           5, 'Historical fiction', 12.50, 433, 2006),
    ('Dune',                           6, 'Science fiction',   14.99, 412, 1965),
    ('The Dispossessed',               1, 'Science fiction',   13.25, 341, 1974),
    ('The Word for World is Forest',   1, 'Science fiction',   10.99, 189, 1976);
```

output

```
INSERT 0 11
```

Author 1 has four books, author 7 has none, and book 11 is never borrowed, which gives later chapters something to find with outer joins.

Example 7: insert the members.

SQL

```sql
INSERT INTO members (full_name, email, joined_on, city) VALUES
    ('Ada Lovelace',      'ada@example.org',      DATE '2024-01-15', 'London'),
    ('Alan Turing',       'alan@example.org',     DATE '2024-02-03', 'Manchester'),
    ('Grace Hopper',      'grace@example.org',    DATE '2024-02-20', 'Arlington'),
    ('Katherine Johnson', 'katherine@example.org',DATE '2024-03-11', 'Hampton'),
    ('Radia Perlman',     'radia@example.org',    DATE '2024-03-25', NULL),
    ('Margaret Hamilton', 'margaret@example.org', DATE '2024-04-08', 'Boston'),
    ('Barbara Liskov',    'barbara@example.org',  DATE '2024-05-02', 'Boston'),
    ('Shafi Goldwasser',  'shafi@example.org',    DATE '2024-06-17', 'New York'),
    ('Tim Berners-Lee',   'tim@example.org',      DATE '2024-07-01', 'London');
```

output

```
INSERT 0 9
```

Member 5 has no city, and member 9 never borrows anything, which is useful when LEFT JOIN examples need a row with no match.

Example 8: insert the loans. Four of them have a NULL return date, so those books are still out.

SQL

```sql
INSERT INTO loans (book_id, member_id, loan_date, return_date) VALUES
    (1, 1, DATE '2025-01-05', DATE '2025-01-19'),
    (3, 1, DATE '2025-01-05', DATE '2025-01-26'),
    (5, 2, DATE '2025-01-12', DATE '2025-02-02'),
    (9, 2, DATE '2025-02-01', DATE '2025-02-15'),
    (2, 3, DATE '2025-02-10', NULL),
    (4, 4, DATE '2025-02-18', DATE '2025-03-04'),
    (6, 5, DATE '2025-03-01', NULL),
    (7, 6, DATE '2025-03-09', DATE '2025-03-23'),
    (8, 7, DATE '2025-03-14', DATE '2025-04-01'),
    (10, 8, DATE '2025-04-02', NULL),
    (5, 3, DATE '2025-04-10', DATE '2025-04-24'),
    (1, 6, DATE '2025-04-15', NULL);
```

output

```
INSERT 0 12
```

Book 1 and book 5 have each been borrowed twice, which matters when a join multiplies rows in Chapter 8.

Example 9: check that everything landed.

SQL

```sql
SELECT
    (SELECT COUNT(*) FROM authors) AS authors,
    (SELECT COUNT(*) FROM books)   AS books,
    (SELECT COUNT(*) FROM members) AS members,
    (SELECT COUNT(*) FROM loans)   AS loans;
```

output

```
 authors | books | members | loans
---------+-------+---------+-------
       7 |    11 |       9 |    12
```

If these four numbers match, every result table in this volume will match too. The bracketed queries here are scalar subqueries; Chapter 9 explains them.

1.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`INSERT INTO books VALUES (...)` with no column list	the values are matched by position, so any column change breaks the insert silently or loudly	always name the columns you fill

Mistake	Why it hurts	Best practice
Typing primary key values by hand	clashes with the identity sequence and produces duplicate-key errors	let `GENERATED ALWAYS AS IDENTITY` assign ids
Storing money in `FLOAT` or `DOUBLE PRECISION`	rounding errors make totals wrong in exact arithmetic-free ways	`NUMERIC(9,2)`
`TEXT` everywhere	no length is enforced, so junk rows pass the door	`VARCHAR(n)` sized to the real data
Deleting an author row that books still reference	the foreign key rejects it with an error	remove or reassign the child rows first, or declare `ON DELETE` behaviour

- Design first, type second: write the tables and keys on paper before typing.
- One fact, one place: never store the author's name inside the books table.
- Keep ids small and meaningless; the identity column does the work.

1.5 Practice exercises

1. Write a `CREATE TABLE` statement for a `publishers` table with an identity primary key `publisher_id`, a `name` column that cannot be NULL, and a `country` column. Then insert two rows without naming the id column.
2. Add a constraint, using `ALTER TABLE`, that refuses any book with zero or negative pages.
3. Insert a new author: Orhan Pamuk, born 1952-06-07, from Turkiye.
4. Which column links `books` to `authors`, and what stops you from inserting a book whose author does not exist?
5. Why does `loans` store `book_id` and `member_id` instead of copies of the book title and the member's name?

Solutions

1. `sql CREATE TABLE publishers (publisher_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, name VARCHAR(80) NOT NULL, country VARCHAR(50));

INSERT INTO publishers (name, country) VALUES ('Faber and Faber', 'United Kingdom'), ('Suhrkamp', 'Germany');

```
    The insert lists columns, so the identity column is filled for you.
2. ```sql
    ALTER TABLE books ADD CONSTRAINT books_pages_positive CHECK (pages > 0);
```

Naming the constraint makes the error message readable later.

3. `sql INSERT INTO authors (first_name, last_name, birth_date, country) VALUES ('Orhan', 'Pamuk', DATE '1952-06-07', 'Turkiye');` This adds an eighth author; drop it again with `DELETE FROM authors WHERE last_name = 'Pamuk';` if you want the exact sample data back.

4. `books.author_id`. It carries a foreign key that references `authors (author_id)`, so the server refuses a book whose `author_id` has no matching author row.
5. Copying the names would store the same fact twice. If a member later corrects the spelling of their name, every copied row goes stale; the id reference stays correct automatically.

1.6 Mini test

1. Multiple choice: which constraint identifies one row of a table uniquely? - a) foreign key - b) primary key - c) check constraint - d) default value
2. True or false: a foreign key column may contain NULL.
3. Multiple choice: which data type is the safe choice for a price? - a) `TEXT` - b) `DOUBLE PRECISION` - c) `NUMERIC` - d) `DATE`
4. Short code question: write the statement that creates a table `tags` with a single identity primary key column named `tag_id`.
5. True or false: SQL keywords must be typed in upper case.
6. Multiple choice: what does `GENERATED ALWAYS AS IDENTITY` do? - a) refuses rows without an id - b) fills the column with sequential numbers automatically - c) makes the column unique but not a key - d) converts the column to text
7. Short answer: name the four tables of the sample schema and the three one-to-many relationships between them.

Answer key and explanations

1. b - the primary key is unique and `NOT NULL`, so it identifies each row.
2. True - a NULL foreign key simply means "no reference recorded"; it is not a violation of the key.
3. c - `NUMERIC` stores amounts exactly, while floating types round.
4. `CREATE TABLE tags (tag_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY);`
5. False - keywords are case-insensitive; `select` and `SELECT` are the same word to the parser.
6. b - the server assigns the next number whenever a row omits the column.
7. `authors`, `books`, `members`, `loans`. Authors write books (one to many), members take loans (one to many), and books are borrowed through loans (one to many).

Chapter 2 - SQL syntax and your first queries

2.1 Summary

- A statement is one instruction; psql sends it to the server when you type the closing semicolon.
- Keywords are case-insensitive, so `select` and `SELECT` are identical to the parser.

- The shape `SELECT ... FROM ...` asks for columns of one table.
- PostgreSQL allows a statement with no `FROM` clause, so `SELECT 1 + 1;` works.
- Whitespace between tokens is ignored, so a query may span as many lines as you like.
- A line comment starts with `--` and a block comment is written `/* ... */`.
- Unquoted identifiers are folded to lower case; double quotes force exact spelling and make `"Title"` a different name from `title`.
- Text values go in single quotes, while double quotes are reserved for identifiers.
- Expressions in the select list are computed once per row and can be renamed with an alias.

2.2 Concepts

The shape of a query

output

```
SELECT    columns and expressions      what to show
FROM      table                        where the rows come from
WHERE     condition                    which rows to keep          (chapter 4)
GROUP BY  columns                      how to bucket the rows     (chapter 9)
HAVING    condition                    which buckets to keep      (chapter 9)
ORDER BY  columns                      the order of the result    (chapter 5)
LIMIT     n                            how many rows to return    (chapter 5)
```

The clauses always appear in this order. PostgreSQL reads them in this order, even though it evaluates `WHERE` before it builds the select list, which is why `SELECT` cannot invent values for rows the `WHERE` clause threw away.

Statements, semicolons and case

SQL

```
SELECT first_name FROM authors;      -- one statement, terminated
```

Inside psql, a statement only runs when you end it with a semicolon; without one the client keeps waiting for more input, which is a friendly way to write a query over several lines. Keyword case is a style choice: this book uses upper case for keywords and lower case for table and column names, which is the most common convention you will meet.

Identifiers and quoting

You write	PostgreSQL sees	Result
`title`	`title`	works, the column exists
`Title`	`title`	works, unquoted names are folded to lower case
`"title"`	`title`	works, quoting an all-lower-case name changes nothing
`"Title"`	`Title`	error, no column is named exactly `Title`

So double quotes are only needed when you insist on mixed-case or reserved names, and they turn a name into an exact match rather than a hint.

Comments

SQL

```sql
-- everything after the two dashes, to the end of the line, is ignored
SELECT title FROM books;  -- trailing comments are allowed too

/* a block comment can span
   several lines and can sit between keywords */
SELECT title FROM books;
```

Comments explain intent, not mechanics: "why this filter exists" is useful, "select the title column" is noise next to a `SELECT title` line.

2.3 Examples

Example 1: every column of the `authors` table.

SQL

```sql
SELECT * FROM authors;
```

output

```
 author_id | first_name |  last_name  | birth_date |    country
-----------+------------+-------------+------------+---------------
         1 | Ursula     | Le Guin     | 1929-10-21 | United States
         2 | Gabriel    | Garcia Marquez | 1927-03-06 | Colombia
         3 | Jane       | Austen      | 1775-12-16 | United Kingdom
         4 | Haruki     | Murakami    | 1949-01-12 | Japan
         5 | Chimamanda | Ngozi Adichie | 1977-09-15 | Nigeria
         6 | Frank      | Herbert     |            | United States
         7 | Kazuo      | Ishiguro    | 1954-11-08 | Japan
```

`*` means "all columns, in table order". Frank Herbert's birth date prints as an empty cell because psql shows NULL that way by default. Use `SELECT *` to explore a table, not in queries an application depends on.

Example 2: name the columns you want.

SQL

```sql
SELECT first_name, last_name, country FROM authors;
```

output

```
 first_name |  last_name  |    country
------------+-------------+---------------
 Ursula     | Le Guin     | United States
 Gabriel    | Garcia Marquez | Colombia
 Jane       | Austen      | United Kingdom
 Haruki     | Murakami    | Japan
 Chimamanda | Ngozi Adichie | Nigeria
 Frank      | Herbert     | United States
 Kazuo      | Ishiguro    | Japan
```

The select list decides both which columns appear and their order in the result.

Example 3: compute a value per row.

SQL

```sql
SELECT first_name,
       last_name,
       LENGTH(last_name) AS name_length
FROM authors;
```

output

```
 first_name |  last_name  | name_length
------------+-------------+-------------
 Ursula     | Le Guin     |           7
 Gabriel    | Garcia Marquez |         13
 Jane       | Austen      |           6
 Haruki     | Murakami    |           8
 Chimamanda | Ngozi Adichie |          14
 Frank      | Herbert     |           7
 Kazuo      | Ishiguro    |           8
```

`LENGTH` counts characters, and the result is renamed with `AS name_length`. Names with a space such as `Garcia Marquez` include the space in the count, so 13 characters for a hyphen-free two-word name is exactly right.

Example 4: build text with the concatenation operator.

SQL

```sql
SELECT first_name || ' ' || last_name AS display_name,
       country
FROM authors;
```

output

```
     display_name     |    country
----------------------+---------------
 Ursula Le Guin       | United States
 Gabriel Garcia Marquez | Colombia
 Jane Austen          | United Kingdom
 Haruki Murakami      | Japan
 Chimamanda Ngozi Adichie | Nigeria
 Frank Herbert        | United States
 Kazuo Ishiguro       | Japan
```

`||` glues strings together and the literal `' '` supplies the space. The alias `display_name` renames the computed column, because an unnamed expression would be reported with an internal name you cannot predict.

Example 5: a statement that touches no table at all.

SQL

```sql
SELECT 2 + 2 AS answer, 'sql' AS greeting;
```

output

```
 answer | greeting
--------+----------
      4 | sql
```

PostgreSQL does not need a `FROM` clause to evaluate expressions, so this is a handy calculator. Note the two different kinds of quotes: `'sql'` is a text value, while `"sql"` would be an identifier and would fail here.

Example 6: comments in place.

SQL

```sql
-- Authors of the sample schema, oldest birth year first is chapter 5;
-- here we simply look at names and countries.
SELECT first_name, last_name   -- two columns are enough
FROM authors;                  -- the schema's smallest table
```

output

```
 first_name |  last_name
------------+-------------
 Ursula     | Le Guin
 Gabriel    | Garcia Marquez
 Jane       | Austen
 Haruki     | Murakami
 Chimamanda | Ngozi Adichie
 Frank      | Herbert
 Kazuo      | Ishiguro
```

The statement is the same as Example 2 with one column dropped; the parser never sees the comments.

Example 7: whitespace is free.

SQL

```sql
SELECT
    first_name,
    last_name
FROM
    authors;
```

output

```
 first_name |  last_name
------------+-------------
 Ursula     | Le Guin
 Gabriel    | Garcia Marquez
 Jane       | Austen
 Haruki     | Murakami
 Chimamanda | Ngozi Adichie
 Frank      | Herbert
 Kazuo      | Ishiguro
```

Newlines and runs of spaces between tokens are ignored, so lay a query out the way that reads best: one clause per line for anything longer than a few words.

2.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Forgetting the semicolon in psql	the prompt silently waits for more input	end every statement with ;

Mistake	Why it hurts	Best practice
`SELECT *` in real applications	returns columns nobody uses and breaks when the table changes	list the columns you need
`"Title"` with double quotes for a value	PostgreSQL reads it as an identifier, so the query fails or misbehaves	single quotes for text, double quotes for names
Leaving `/*` unclosed	everything after it, including later statements, is swallowed	close every block comment on the same nesting
Cramming a whole query on one line	unreadable and impossible to debug	one clause per line, keywords in upper case
Guessing column names	error messages cost time	`\d books` in psql shows every column and its type

- Keywords upper case, names lower case: the eye finds structure faster.
- Comment the reason for a filter or a join, never the syntax itself.
- Run a query as soon as you write it; SQL feedback is instant and free.

2.5 Practice exercises

1. Write a query that returns `first_name` and `last_name` for every author.
2. Why does `SELECT "Title" FROM books;` fail while `SELECT title FROM books;` works?
3. Write a statement that returns the number of pages of book 1 plus 100 as a single column named `longer`.
4. Write a statement that calculates `17 * 3` without using any table.
5. In PostgreSQL, what does `SELECT 10 / 4;` return, and how do you make it return 2.5?

Solutions

1. `sql SELECT first_name, last_name FROM authors;`
2. Double quotes always denote an identifier, and unquoted identifiers are folded to lower case, so `"Title"` asks for a column spelled exactly `Title`. No such column exists; the real name is `title`.
3. `sql SELECT pages + 100 AS longer FROM books WHERE book_id = 1;` `WHERE` is taught in Chapter 4; the answer is 283. If you wrote it before Chapter 4, run `SELECT 183 + 100 AS longer;` which needs no table.
4. `sql SELECT 17 * 3;` It returns 51; the column has no name of its own unless you alias it.
5. `10 / 4` divides two integers, so PostgreSQL performs integer division and returns `2`. Write `SELECT ROUND(10 / 4.0, 1);` to get `2.5`; the decimal literal forces numeric division, and `ROUND` trims the extra decimals.

2.6 Mini test

1. Multiple choice: which token tells psql that a statement is complete? - a) `.` - b) `;` - c) `GO` - d) `/`

2. True or false: `SELECT` and `select` are the same keyword to PostgreSQL.
3. Multiple choice: what starts a comment that runs to the end of the line? - a) `#` - b) `//` - c) `--` - d) `/*`
4. Short code question: write a query returning the `title` and `genre` of every book.
5. True or false: `SELECT 1;` is invalid in PostgreSQL because it has no `FROM` clause.
6. Multiple choice: which of these is a text value rather than an identifier? - a) `'hello'` - b) `"hello"` - c) `(hello)` - d) `hello`
7. Short answer: what does PostgreSQL do with the spaces and newlines between keywords?

Answer key and explanations

1. b - the semicolon terminates the statement; without it psql waits for more input.
2. True - keywords are case-insensitive.
3. c - `--` comments out the rest of the line.
4. `SELECT title, genre FROM books;`
5. False - PostgreSQL evaluates expressions without a `FROM` clause, a convenience some other databases do not offer.
6. a - single quotes make a string constant; double quotes make an identifier.
7. It ignores them. A query may be broken across lines and tokens may be separated by any amount of whitespace.

Chapter 3 - Choosing columns: aliases and DISTINCT

3.1 Summary

- The select list is a projection: it decides which columns and expressions appear in the result.
- An expression in the select list is evaluated once per row and can be renamed with an alias.
- A column alias is written `AS name`, and the `AS` keyword itself is optional.
- Double quotes around an alias allow spaces and capitals, as in `AS "Book title"`.
- A table alias shortens qualified references such as `b.title` and is written `FROM books AS b`.
- `DISTINCT` removes duplicate rows from the result set; it never changes the stored table.
- `DISTINCT` compares the whole selected row, so `SELECT DISTINCT author_id, genre` keeps unique pairs.
- Function arguments can also take `DISTINCT`, as in `COUNT(DISTINCT genre)`.
- The result of any query is itself a table-shaped set of rows with its own column names.

3.2 Concepts

Projection: choosing what you see

```
    books (11 rows, 7 columns)
         |
         |  SELECT title, price ...
         v
    result (11 rows, 2 columns)   <- a projection of the table
```

Projection never adds or removes rows; it only narrows and reshapes the columns. Expressions in the list turn each row into new values, computed on the fly.

Column aliases

Form	Example	Notes
With `AS`	`title AS book_title`	the readable form used in this book
Without `AS`	`title book_title`	legal, but harder to spot
Quoted	`title AS "Book title"`	needed for spaces, capitals or keywords
From an expression	`pages * 2 AS double_pages`	gives a computed column its name

An alias exists only in the result; the table column keeps its original name.

Table aliases

```
SELECT b.title, b.price          b is an alias for books
FROM books AS b;                 the prefix b. removes any doubt
```

Once a query names two tables (Chapter 8) a prefix is the only way to say which table a column belongs to. With one table the prefix is still useful when the same column name exists in two tables you will join later.

DISTINCT

```
    before: Fiction, Fiction, Romance, Science fiction, ...
    after : Fiction, Romance, Science fiction, ...
```

`SELECT DISTINCT` looks at the complete selected row and keeps the first occurrence of each unique combination. It is a reporting tool: if you need `DISTINCT` to make your data look right, the cause is usually duplicated data or a join that multiplies rows.

3.3 Examples

Example 1: a table alias with qualified column names.

SQL

```sql
SELECT a.first_name, a.last_name
FROM authors AS a;
```

output

```
 first_name |  last_name
------------+-------------
 Ursula     | Le Guin
 Gabriel    | Garcia Marquez
 Jane       | Austen
 Haruki     | Murakami
 Chimamanda | Ngozi Adichie
 Frank      | Herbert
 Kazuo      | Ishiguro
```

`authors AS a` names the table `a` for the rest of the statement. The `AS` before a table alias is optional in PostgreSQL, exactly as it is for columns.

Example 2: an expression with an alias, using the table alias as prefix.

SQL

```sql
SELECT b.title,
       b.pages,
       ROUND(b.pages / 2.0, 1) AS half_pages
FROM books AS b;
```

output

```
            title            | pages | half_pages
-----------------------------+-------+------------
 A Wizard of Earthsea        |   183 |       91.5
 The Left Hand of Darkness   |   304 |      152.0
 One Hundred Years of Solitude |  417 |      208.5
 Love in the Time of Cholera |   348 |      174.0
 Pride and Prejudice         |   279 |      139.5
 Norwegian Wood              |   296 |      148.0
 Kafka on the Shore          |   505 |      252.5
 Half of a Yellow Sun        |   433 |      216.5
 Dune                        |   412 |      206.0
 The Dispossessed            |   341 |      170.5
 The Word for World is Forest |   189 |       94.5
```

Dividing by `2.0` forces numeric division, so 183 pages become 91.5 instead of the integer 91. `ROUND(x, 1)` keeps one decimal, which suppresses the long tail of decimals numeric division would otherwise print.

Example 3: quoted aliases with spaces.

SQL

```sql
SELECT first_name AS "First name", last_name AS "Surname"
FROM authors;
```

output

```
 First name |  Surname
------------+-------------
 Ursula     | Le Guin
 Gabriel    | Garcia Marquez
 Jane       | Austen
 Haruki     | Murakami
 Chimamanda | Ngozi Adichie
 Frank      | Herbert
 Kazuo      | Ishiguro
```

Without the double quotes the header would be reported as `First` and `name` would break the statement, so quoting is what makes a two-word header legal.

Example 4: the alias without `AS`, to see the shortest legal form.

SQL

```sql
SELECT first_name || ' ' || last_name author
FROM authors;
```

output

```
          author
-----------------------
 Ursula Le Guin
 Gabriel Garcia Marquez
 Jane Austen
 Haruki Murakami
 Chimamanda Ngozi Adichie
 Frank Herbert
 Kazuo Ishiguro
```

The statement is identical to the `AS` version; leaving `AS` out saves four characters and costs a little clarity.

Example 5: `DISTINCT` on one column.

SQL

```sql
SELECT DISTINCT country FROM authors;
```

output

```
    country
--------------
 Colombia
 Japan
 Nigeria
 United Kingdom
 United States
```

Japan is stored twice, once for Murakami and once for Ishiguro, and `DISTINCT` reports it once. The order of the rows is not guaranteed by the query; Chapter 5 shows how to pin it down with `ORDER BY`.

Example 6: `DISTINCT` on another column with more repeats.

SQL

```sql
SELECT DISTINCT genre FROM books;
```

output

```
      genre
---------------
 Fantasy
 Science fiction
 Magical realism
 Romance
 Fiction
 Historical fiction
```

Example 7: `DISTINCT` across two columns.

SQL

```sql
SELECT DISTINCT author_id, genre FROM books;
```

output

```
 author_id |      genre
-----------+----------------
         1 | Fantasy
         1 | Science fiction
         2 | Magical realism
         2 | Romance
         3 | Romance
         4 | Fiction
         5 | Historical fiction
         6 | Science fiction
```

Eleven books collapse to eight rows, because author 1 owns three Science fiction titles and author 4 owns two Fiction titles.

Example 8: `DISTINCT` inside a function call.

SQL

```sql
SELECT COUNT(DISTINCT genre) AS distinct_genres
FROM books;
```

output

```
 distinct_genres
-----------------
               6
```

The `DISTINCT` here applies to the function's argument only, so it counts unique genre values rather than rows. Aggregates are covered fully in Chapter 9.

3.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Using `DISTINCT` to make results "look right"	hides duplicated data or a multiplying join	fix the cause, then drop `DISTINCT`
Expecting `DISTINCT` to work column by column	it compares the whole row	select exactly the columns that define a duplicate

Mistake	Why it hurts	Best practice
Using an alias in `WHERE`	the alias does not exist yet when `WHERE` runs	repeat the expression, or wrap the query in a subquery
Double-quoting every alias	forces exact spelling in every later tool	quote only when the alias needs spaces or capitals
`SELECT *` and then deduplicating	fetches columns you throw away	ask for the columns you actually need

- Alias every computed column; an unnamed expression gets an unpredictable header.
- Keep aliases short but honest: `half_pages` beats `h`.
- `DISTINCT` is cheap to try, so experiment freely in psql.

3.5 Practice exercises

1. Write a query that returns each author as one column headed `Full name`.
2. Write a query that lists every country exactly once.
3. How many rows does `SELECT DISTINCT author_id, genre FROM books;` return, and why that number?
4. Rewrite `SELECT title AS t FROM books;` without the `AS` keyword.
5. Why can `WHERE` not filter on an alias defined in the select list of the same query?

Solutions

1. `sql SELECT first_name || ' ' || last_name AS "Full name" FROM authors;` The quotes are needed because the header contains a space.
2. `sql SELECT DISTINCT country FROM authors;` Five rows come back: Colombia, Japan, Nigeria, United Kingdom, United States.
3. Eight rows. The eleven books share author and genre combinations, so the pairs `(1, Science fiction)` and `(4, Fiction)` collapse to one row each.
4. `sql SELECT title t FROM books;` Both forms are valid PostgreSQL; `AS` is pure documentation.
5. The select list is conceptually evaluated after `WHERE`, so the alias is not a name the filter can see. Either repeat the full expression inside `WHERE` or put the query in a subquery and filter outside it.

3.6 Mini test

1. Multiple choice: which keyword removes duplicate rows from a result set? - a) `UNIQUE` - b) `DISTINCT` - c) `ONLY` - d) `DEDUPE`
2. True or false: `DISTINCT` is applied to each selected column independently.
3. Multiple choice: which of these is a correct column alias? - a) `title == t` - b) `title AS t` - c) `title -> t` - d) `title @ t`

4. Short code question: write a query that returns the title of every book with the header `Book title`.
5. True or false: a table alias must be introduced with the `AS` keyword.
6. Multiple choice: what does `COUNT(DISTINCT genre)` return? - a) the number of rows - b) the number of distinct genre values - c) the number of NULL genres - d) the number of books per genre
7. Short answer: does `DISTINCT` modify the data stored in the table?

Answer key and explanations

1. b - `DISTINCT` collapses duplicate result rows; `UNIQUE` is a table constraint, not a query keyword.
2. False - it compares the entire selected row as one unit.
3. b - `AS` introduces a column alias; the other symbols are not SQL.
4. `SELECT title AS "Book title" FROM books;` (quotes are optional only if the header has no spaces or capitals).
5. False - `AS` is optional: `FROM books b` works exactly like `FROM books AS b`.
6. b - it counts unique non-NULL values of the argument, which is 6 here.
7. No. It only shapes the result set you are looking at; the table keeps every row it had.

Chapter 4 - Filtering rows with WHERE

4.1 Summary

- `WHERE` keeps the rows whose condition is true and discards all the others.
- Comparison operators are `=`, `<>`, `<`, `<=`, `>` and `>=`.
- `AND` requires every condition to be true, `OR` requires at least one, and `NOT` reverses a condition.
- `AND` binds more tightly than `OR`, so parentheses make mixed conditions explicit and safe.
- `IN` tests membership in a list and `BETWEEN` tests a closed range; both are shorthands for chains of `AND`.
- `LIKE` matches text patterns, where `%` stands for any run of characters and `_` for exactly one.
- `LIKE` is case-sensitive in PostgreSQL; `ILIKE` is its case-insensitive sibling.
- A comparison with `NULL` never comes out true, so NULL is tested with `IS NULL` or `IS NOT NULL`.
- Text values are written in single quotes and numbers without quotes.

4.2 Concepts

The filter in the pipeline

output

```
FROM books          every row of the table
   |
WHERE price > 13   keep only the rows whose condition is true
   |
SELECT title, ...  shape the surviving rows (chapter 3)
```

The condition is evaluated once per row, using only that row's columns; a row either passes or it does not.

Comparison operators

Operator	Meaning	Example
=	equal to	`genre = 'Romance'`
<>	not equal to	`genre <> 'Romance'`
< <=	less than, or equal	`price <= 12`
> >=	greater than, or equal	`published_year >= 1970`

Text and dates compare with the same operators; `'A' < 'B'` is true, and `DATE '2025-01-05' < DATE '2025-02-01'` is true because PostgreSQL compares dates as days.

Boolean logic

output

```
   a     |   b    | a AND b | a OR b
---------+--------+---------+--------
 true    | true   | true    | true
 true    | false  | false   | true
 false   | true   | false   | true
 false   | false  | false   | false
```

`NOT true` is false and `NOT false` is true. Because `AND` binds more tightly than `OR`, the condition

SQL

```sql
WHERE genre = 'Romance' OR genre = 'Fiction' AND price > 12
```

means "all Romance books, plus Fiction books above 12". Parentheses are the cheapest debugging tool in SQL:

SQL

```sql
WHERE (genre = 'Romance' OR genre = 'Fiction') AND price > 12
```

IN and BETWEEN

SQL

```sql
WHERE published_year IN (1813, 1965, 1967)   -- same as = 1813 OR = 1965 OR = 1967
WHERE price BETWEEN 10 AND 13               -- same as >= 10 AND <= 13, both ends closed
```

Both accept `NOT` in front of them: `NOT IN` and `NOT BETWEEN` keep the rows the positive form would reject.

Pattern matching with LIKE

Pattern	Matches	Example in this schema
`%`	any sequence of characters, including none	`'%the%'` finds titles containing "the"
`_`	exactly one character	`'_une'` matches `Dune`
`'The %'`	titles starting with "The "	`The Left Hand of Darkness`

`LIKE` compares case-sensitively, so `'%the%'` misses the capitalised "The". `ILIKE` folds case, and `NOT LIKE` inverts the test. To search for a literal `%` or _, escape it with a backslash: `LIKE '50\%'`.

NULL is not a value

`price = NULL`, `price <> NULL` and `price > NULL` all evaluate to NULL, which is not true, so the row is dropped. The correct tests are `IS NULL` and `IS NOT NULL`, which never return NULL.

4.3 Examples

Example 1: one comparison.

SQL

```sql
SELECT book_id, title, price
FROM books
WHERE price > 13;
```

output

```
 book_id |           title            | price
---------+----------------------------+-------
       2 | The Left Hand of Darkness  | 14.50
       3 | One Hundred Years of Solitude | 15.20
       4 | Love in the Time of Cholera | 13.75
       9 | Dune                       | 14.99
      10 | The Dispossessed           | 13.25
```

Five books cost more than 13. The book priced exactly `13.00` is excluded, because `>` is strict.

Example 2: two conditions with `AND`.

SQL

```sql
SELECT book_id, title, genre, published_year
FROM books
WHERE genre = 'Science fiction' AND published_year < 1970;
```

output

```
 book_id |           title            |      genre      | published_year
---------+----------------------------+-----------------+----------------
       2 | The Left Hand of Darkness  | Science fiction |           1969
       9 | Dune                       | Science fiction |           1965
```

Both conditions must hold, so only two of the four Science fiction titles survive.

Example 3: `AND` beats `OR`.

SQL

```sql
SELECT book_id, title, genre, price
FROM books
WHERE genre = 'Romance' OR genre = 'Fiction' AND price > 12;
```

output

```
 book_id |           title            |  genre   | price
---------+----------------------------+----------+-------
       4 | Love in the Time of Cholera | Romance  | 13.75
       5 | Pride and Prejudice        | Romance  |  9.99
       7 | Kafka on the Shore         | Fiction  | 13.00
```

The condition means "Romance, or Fiction priced above 12", so cheap Romance is included and cheap Fiction is not. Adding parentheses changes the meaning:

SQL

```sql
SELECT book_id, title, genre, price
FROM books
WHERE (genre = 'Romance' OR genre = 'Fiction') AND price > 12;
```

output

```
 book_id |           title            |  genre   | price
---------+----------------------------+----------+-------
       4 | Love in the Time of Cholera | Romance  | 13.75
       7 | Kafka on the Shore         | Fiction  | 13.00
```

Now both genres must clear the same price bar, and book 5 drops out.

Example 4: `IN` with a list of years.

SQL

```sql
SELECT book_id, title, published_year
FROM books
WHERE published_year IN (1813, 1965, 1967);
```

output

```
 book_id |           title            | published_year
---------+----------------------------+----------------
       3 | One Hundred Years of Solitude |           1967
       5 | Pride and Prejudice        |           1813
       9 | Dune                       |           1965
```

`IN` reads much better than three `OR` conditions and gives the same rows.

Example 5: `BETWEEN` with a closed range.

SQL

```sql
SELECT book_id, title, price
FROM books
WHERE price BETWEEN 10 AND 13;
```

output

```
 book_id |           title            | price
---------+----------------------------+-------
       1 | A Wizard of Earthsea       | 12.99
       6 | Norwegian Wood             | 11.40
       7 | Kafka on the Shore         | 13.00
       8 | Half of a Yellow Sun       | 12.50
      11 | The Word for World is Forest | 10.99
```

Both ends are included, so the books at `10.99` and `13.00` are kept.

Example 6: `NOT IN` with `COUNT` for a compact answer.

SQL

```sql
SELECT COUNT(*) AS books_not_by_author_1
FROM books
WHERE author_id NOT IN (1);
```

output

```
 books_not_by_author_1
-----------------------
                     7
```

Author 1 wrote four of the eleven books, so seven remain. Remember that a NULL inside the `IN` list would make `NOT IN` return no rows at all, because the comparison with NULL is unknown rather than true or false.

Example 7: `LIKE` against `ILIKE`.

SQL

```sql
SELECT book_id, title
FROM books
WHERE title LIKE '%the%';
```

output

```
 book_id |           title
---------+------------------------
       4 | Love in the Time of Cholera
       7 | Kafka on the Shore
```

SQL

```sql
SELECT book_id, title
FROM books
WHERE title ILIKE '%the%';
```

output

```
 book_id |           title
---------+------------------------
       2 | The Left Hand of Darkness
       4 | Love in the Time of Cholera
       7 | Kafka on the Shore
      10 | The Dispossessed
      11 | The Word for World is Forest
```

The case-sensitive `LIKE` sees only the lower-case "the" inside two titles; the case-insensitive `ILIKE` also catches the three titles that begin with "The".

Example 8: the single-character wildcard and a prefix.

SQL

```sql
SELECT book_id, title FROM books WHERE title LIKE '_une';
```

output

```
 book_id | title
---------+------
       9 | Dune
```

_ stands for exactly one character, so `_une` matches `Dune` and nothing else.

SQL

```sql
SELECT book_id, title FROM books WHERE title LIKE 'The %';
```

output

```
 book_id |           title
---------+------------------------
       2 | The Left Hand of Darkness
      10 | The Dispossessed
      11 | The Word for World is Forest
```

`The %` requires the capitalised prefix followed by a space and then anything.

Example 9: finding the rows where a value is missing.

SQL

```sql
SELECT loan_id, book_id, member_id, loan_date
FROM loans
WHERE return_date IS NULL;
```

output

```
 loan_id | book_id | member_id | loan_date
---------+---------+-----------+------------
       5 |       2 |         3 | 2025-02-10
       7 |       6 |         5 | 2025-03-01
      10 |      10 |         8 | 2025-04-02
      12 |       1 |         6 | 2025-04-15
```

These four loans are still out. The tempting shortcut

SQL

```sql
SELECT COUNT(*) AS wrong FROM loans WHERE return_date = NULL;
```

returns `0`, never four, because `= NULL` is always unknown.

4.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`column = NULL`	the comparison is unknown, so no row passes	`IS NULL` / `IS NOT NULL`
Text value without quotes	PostgreSQL reads it as a column name and errors	`'Science fiction'`
Relying on `AND`/`OR` precedence	`a OR b AND c` means `a OR (b AND c)`	parentheses around every mixed group
Expecting `LIKE` to ignore case	it is case-sensitive	`ILIKE`, or `lower(column) LIKE ...`
`BETWEEN` on strings for dates	the format decides the cast and can surprise	`DATE '...'` literals
`NOT IN` with a NULL inside the list	the whole condition becomes unknown and returns nothing	filter NULLs out of the list first

- Write one condition per line; long `WHERE` clauses are read more often than they are written.
- Test the boundary values: `>`, `>=`, `<`, `<=` differ exactly at the edge.
- When a filter returns nothing, suspect NULL before suspecting the data.

4.5 Practice exercises

1. List the titles and prices of the books priced between 12 and 14 inclusive.
2. Which members joined on or after 2024-04-01? Give their names.
3. List the books written by author 1 or author 2 that are not Science fiction.
4. Why does `WHERE country = NULL` return no rows while `WHERE country IS NULL` works?
5. How many loans are still out, and what are their loan ids?

Solutions

1. `sql SELECT title, price FROM books WHERE price BETWEEN 12 AND 14;` Six rows: books 1, 2, 4, 7, 8 and 10.
2. `sql SELECT full_name FROM members WHERE joined_on >= DATE '2024-04-01';` Margaret Hamilton, Barbara Liskov, Shafi Goldwasser and Tim Berners-Lee.
3. `sql SELECT title FROM books WHERE author_id IN (1, 2) AND genre <> 'Science fiction';` A Wizard of Earthsea, One Hundred Years of Solitude and Love in the Time of Cholera.
4. `= NULL` is evaluated as unknown, and `WHERE` keeps only rows where the condition is true; unknown is neither true nor false. `IS NULL` is the operator designed to test for NULL.
5. Four loans, ids 5, 7, 10 and 12, because their `return_date` is NULL.

4.6 Mini test

1. Multiple choice: which syntax correctly tests a column for NULL? - a) `col = NULL` - b) `col IS NULL` - c) `col == NULL` - d) `col ~ NULL`

2. True or false: `AND` binds more tightly than `OR`.
3. Multiple choice: in a `LIKE` pattern, what does `%` stand for? - a) exactly one character - b) any sequence of zero or more characters - c) one digit - d) a literal percent sign only
4. Short code question: write a `WHERE` condition that keeps books priced at most 12.
5. True or false: `LIKE 'Dune'` matches only the exact title `Dune`.
6. Multiple choice: what does `WHERE return_date = NULL` return? - a) the open loans - b) no rows at all - c) an error - d) every row
7. Short answer: write the correct condition for finding the loans that are still out.

Answer key and explanations

1. b - `IS NULL` is the only NULL test; `= NULL` is always unknown.
2. True - so `a OR b AND c` is read as `a OR (b AND c)` unless you add parentheses.
3. b - `%` matches any run of characters including none; _ is the single-character wildcard.
4. `WHERE price <= 12;` (or `WHERE NOT price > 12;`, though the first form is the natural one).
5. True - without wildcards `LIKE` is an exact, case-sensitive match.
6. b - the comparison evaluates to unknown, and unknown rows are discarded.
7. `WHERE return_date IS NULL;`

Chapter 5 - Sorting and limiting results

5.1 Summary

- `ORDER BY` sorts the result set by one or more sort keys.
- Each key is sorted `ASC` (ascending, the default) or `DESC`.
- With several keys, the second one only breaks ties left by the first, and so on left to right.
- Text sorts by collation, numbers by value, and dates in chronological order.
- In PostgreSQL, `ASC` puts NULLs last and `DESC` puts them first unless you say otherwise.
- `NULLS FIRST` and `NULLS LAST` choose the placement of NULLs explicitly, key by key.
- `LIMIT n` keeps only the first n rows of the result; `OFFSET n` skips n rows first.
- `LIMIT` without `ORDER BY` returns an arbitrary slice, so the two belong together.
- SQL Server expresses the same idea with `TOP`, which is worth recognising when you read other people's SQL.

5.2 Concepts

ORDER BY and its keys

SQL

```sql
SELECT title, genre, price
FROM books
ORDER BY genre ASC, price DESC;
```

The keys are applied left to right: rows are grouped by `genre`, and inside each genre the prices run from high to low. Any column of the table may be a key, whether or not it appears in the select list, and an expression such as `LENGTH(title)` works as a key too.

Where NULL goes

Sort	Default NULL placement	How to force the other
`ASC`	last	`ORDER BY col ASC NULLS FIRST`
`DESC`	first	`ORDER BY col DESC NULLS LAST`

PostgreSQL treats NULL as larger than every value, which is why ascending order files it at the end and descending order files it at the front.

LIMIT and OFFSET

output

```
    ORDER BY book_id          rows 1 .. 11 in a fixed order
    OFFSET 4  LIMIT 4         skip 4, keep 4  -> rows 5, 6, 7, 8
```

Paging is arithmetic: page `p` with page size `s` needs `OFFSET (p - 1) * s LIMIT s`. Page 2 of 4, for example, is `OFFSET 4 LIMIT 4`.

> Other spellings: SQL Server uses `TOP (n)` right after `SELECT`, and the ANSI standard writes `OFFSET n ROWS FETCH NEXT m ROWS ONLY`, which PostgreSQL also understands. `LIMIT`/`OFFSET` is the PostgreSQL shorthand you will use here.

5.3 Examples

Example 1: the cheapest books.

SQL

```sql
SELECT book_id, title, price
FROM books
ORDER BY price ASC
LIMIT 5;
```

output

```
 book_id |           title            | price
---------+----------------------------+-------
       5 | Pride and Prejudice        |  9.99
      11 | The Word for World is Forest | 10.99
       6 | Norwegian Wood             | 11.40
       8 | Half of a Yellow Sun       | 12.50
       1 | A Wizard of Earthsea       | 12.99
```

`ASC` is the default, so `ORDER BY price` would give the same rows. Without the `LIMIT`, the other six books would simply follow.

Example 2: the most expensive books.

SQL

```sql
SELECT book_id, title, price
FROM books
ORDER BY price DESC
LIMIT 5;
```

output

```
 book_id |           title            | price
---------+----------------------------+-------
       3 | One Hundred Years of Solitude | 15.20
       9 | Dune                       | 14.99
       2 | The Left Hand of Darkness  | 14.50
       4 | Love in the Time of Cholera | 13.75
      10 | The Dispossessed           | 13.25
```

`DESC` reverses the comparison, and `LIMIT` then keeps the top of that order.

Example 3: two sort keys.

SQL

```sql
SELECT book_id, title, genre, price
FROM books
ORDER BY genre ASC, price ASC;
```

output

```
 book_id |           title            |      genre      | price
---------+----------------------------+-----------------+-------
       1 | A Wizard of Earthsea       | Fantasy         | 12.99
       6 | Norwegian Wood             | Fiction         | 11.40
       7 | Kafka on the Shore         | Fiction         | 13.00
       8 | Half of a Yellow Sun       | Historical fiction | 12.50
       3 | One Hundred Years of Solitude | Magical realism | 15.20
       5 | Pride and Prejudice        | Romance         |  9.99
       4 | Love in the Time of Cholera | Romance         | 13.75
      11 | The Word for World is Forest | Science fiction | 10.99
      10 | The Dispossessed           | Science fiction | 13.25
       2 | The Left Hand of Darkness  | Science fiction | 14.50
       9 | Dune                       | Science fiction | 14.99
```

The first key groups by genre; the second orders titles inside each group, so the two Romance books appear cheapest first.

Example 4: sorting a column that contains NULL.

SQL

```sql
SELECT first_name, last_name, birth_date
FROM authors
ORDER BY birth_date ASC;
```

output

```
 first_name |  last_name  | birth_date
------------+-------------+------------
 Jane       | Austen      | 1775-12-16
 Gabriel    | Garcia Marquez | 1927-03-06
 Ursula     | Le Guin     | 1929-10-21
 Haruki     | Murakami    | 1949-01-12
 Kazuo      | Ishiguro    | 1954-11-08
 Chimamanda | Ngozi Adichie | 1977-09-15
 Frank      | Herbert     |
```

Ascending order keeps NULLs at the end by default, and psql prints Frank Herbert's missing birth date as an empty cell.

Example 5: forcing NULLs to the other end.

SQL

```sql
SELECT first_name, last_name, birth_date
FROM authors
ORDER BY birth_date DESC NULLS FIRST;
```

output

```
 first_name |  last_name  | birth_date
------------+-------------+------------
 Frank      | Herbert     |
 Chimamanda | Ngozi Adichie | 1977-09-15
 Kazuo      | Ishiguro    | 1954-11-08
 Haruki     | Murakami    | 1949-01-12
 Ursula     | Le Guin     | 1929-10-21
 Gabriel    | Garcia Marquez | 1927-03-06
 Jane       | Austen      | 1775-12-16
```

The unknown birth date sorts first now, which is what you want when the missing values are the ones you are hunting.

Example 6: paging with `OFFSET` and `LIMIT`.

SQL

```sql
SELECT book_id, title
FROM books
ORDER BY book_id
OFFSET 4 LIMIT 4;
```

output

```
 book_id |           title
---------+-----------------------
       5 | Pride and Prejudice
       6 | Norwegian Wood
       7 | Kafka on the Shore
       8 | Half of a Yellow Sun
```

SQL

```sql
SELECT book_id, title
FROM books
ORDER BY book_id
OFFSET 8 LIMIT 4;
```

output

```
 book_id |           title
---------+------------------------
       9 | Dune
      10 | The Dispossessed
      11 | The Word for World is Forest
```

The second statement asks for four rows but only three remain, which is not an error. Always pair `OFFSET` with `ORDER BY`, or the "page" is arbitrary.

Example 7: sorting by an expression.

SQL

```sql
SELECT book_id, title, LENGTH(title) AS title_length
FROM books
ORDER BY LENGTH(title) DESC, book_id
LIMIT 3;
```

output

```
 book_id |           title            | title_length
---------+----------------------------+--------------
       3 | One Hundred Years of Solitude |           29
      11 | The Word for World is Forest |           28
       4 | Love in the Time of Cholera |           27
```

The tie-breaker `book_id` makes the order deterministic when two titles share a length. Unlike `WHERE`, `ORDER BY` may also use an alias from the select list, so `ORDER BY title_length DESC` would work here too.

5.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`LIMIT` without `ORDER BY`	the rows you get are an arbitrary set	sort first, then limit
Expecting `DESC` to keep NULLs last	PostgreSQL files NULLs first in descending order	`NULLS LAST` when the position matters
`ORDER BY 2` with a positional number	the number refers to the select list, so changing the query changes the sort	name the column or its alias
Sorting on a column that is missing from the result and wondering where it went	sort keys need not be selected, which is legal but confusing	select what you sort by in reports
Paging with `OFFSET (page * size)`	skips one page too many	`OFFSET (page - 1) * size`

- Add a unique tie-breaker such as the primary key to every sort that feeds a report, so the same query always returns the same order.
- Sort in the database, not in the application: the server is faster and you avoid shipping rows you never show.

5.5 Practice exercises

1. Write a query returning the three most expensive books with their prices.
2. List the members ordered by the day they joined, newest first.
3. Write the statement for page 3 of the book list when the pages are four rows and the order is by title.
4. Sort the authors by birth date with the unknown birth date listed first.
5. What does `ORDER BY 2` mean, and why is naming the column safer?

Solutions

1. `sql SELECT title, price FROM books ORDER BY price DESC LIMIT 3;` One Hundred Years of Solitude 15.20, Dune 14.99, The Left Hand of Darkness 14.50.
2. `sql SELECT full_name, joined_on FROM members ORDER BY joined_on DESC;` Tim Berners-Lee first (2024-07-01), Ada Lovelace last (2024-01-15).
3. `sql SELECT title FROM books ORDER BY title OFFSET 8 LIMIT 4;` The three rows returned are The Dispossessed, The Left Hand of Darkness and The Word for World is Forest.
4. `sql SELECT first_name, last_name, birth_date FROM authors ORDER BY birth_date DESC NULLS FIRST;` Frank Herbert's unknown birth date appears first.
5. The number refers to the second column of the select list. If someone adds a column at the front, the sort silently changes meaning; a name survives edits.

5.6 Mini test

1. Multiple choice: which sort direction is the default? - a) `DESC` - b) `ASC` - c) the order of insertion - d) alphabetical
2. True or false: in PostgreSQL, `ORDER BY price DESC` places NULL prices last.
3. Multiple choice: which clause keeps only the first rows of a result? - a) `TOP` - b) `LIMIT` - c) `FIRST` - d) `HEAD`
4. Short code question: write a query returning the three cheapest books.
5. True or false: `LIMIT 5` without `ORDER BY` returns the five rows that were inserted first.
6. Multiple choice: with 11 books ordered by `book_id`, which rows does `OFFSET 4 LIMIT 4` return? - a) books 1 to 4 - b) books 4 to 7 - c) books 5 to 8 - d) books 8 to 11
7. Short answer: may `ORDER BY` use an alias defined in the select list?

Answer key and explanations

1. b - ascending order is the default; `ASC` may be written or left out.
2. False - descending order files NULLs first, so use `NULLS LAST` to change that.
3. b - `LIMIT n` keeps the first n rows; SQL Server spells the same idea `TOP`.
4. `sql SELECT title FROM books ORDER BY price LIMIT 3;` Pride and Prejudice, The Word for World is Forest and Norwegian Wood.

5. False - without `ORDER BY` the rows have no defined order, so any five rows may come back.
6. c - `OFFSET 4` skips rows 1 to 4 and `LIMIT 4` keeps the next four.
7. Yes. `ORDER BY` is evaluated after the select list, so aliases are visible there, unlike in `WHERE`.

Chapter 6 - Changing data: INSERT, UPDATE and DELETE

6.1 Summary

- `INSERT` adds rows; every column you omit receives its default value or NULL.
- One `INSERT` statement can add many rows through a comma-separated `VALUES` list.
- `UPDATE` changes existing rows, and almost always needs a `WHERE` to say which ones.
- `DELETE` removes rows, and almost always needs a `WHERE` for the same reason.
- Without `WHERE`, `UPDATE` and `DELETE` affect every row in the table, with no warning.
- A transaction groups statements so that they either all take effect or none do, written `BEGIN` ... `COMMIT` or `ROLLBACK`.
- `INSERT INTO ... SELECT` copies the result of a query into a table.
- `DELETE` removes rows while `DROP TABLE` removes the table and its contents for good.
- Values that come from users must travel as query parameters, never glued into the SQL text.

6.2 Concepts

The three data-changing statements

Statement	Effect	Shape
`INSERT`	adds rows	`INSERT INTO t (cols) VALUES (...), (...);`
`UPDATE`	changes values in existing rows	`UPDATE t SET col = expr WHERE cond;`
`DELETE`	removes rows	`DELETE FROM t WHERE cond;`

`UPDATE` reads the old row, computes the new values from it, and writes them back, so `SET price = price * 1.1` is the natural way to raise prices.

Transactions

output

```
BEGIN;                start a block of work
   UPDATE ...;
   DELETE ...;
COMMIT;               make it permanent      -- or --
ROLLBACK;             undo everything since BEGIN
```

Inside a transaction, other users cannot see your half-finished work, and `ROLLBACK` restores the exact previous state, which makes it the safest sandbox for practising an `UPDATE` you are unsure about. psql runs each statement in its own transaction unless you open one yourself.

Copying rows between tables

`INSERT INTO target SELECT ... FROM source;` appends whole rows produced by a query, and `CREATE TABLE target (LIKE source);` builds an empty table with the same columns first. `SELECT INTO`, the older spelling, still works in PostgreSQL as `SELECT ... INTO target FROM ...;`.

Keep values out of the SQL text

output

```
dangerous, user text glued into the statement:
  ... WHERE email = '' OR '1' = '1' --;

safe, the value sent as a parameter:
  ... WHERE email = $1
```

When text from a form is pasted into a query, a crafted input can close the quote and append its own statement, a attack known as SQL injection. The fix is to send values separately from the statement, which every driver supports and which PostgreSQL also exposes directly through `PREPARE` with `$1`, `$2`, ...

6.3 Examples

Example 1: add one member and count.

SQL

```sql
INSERT INTO members (full_name, email, joined_on, city)
VALUES ('Linus Pauling', 'linus@example.org', DATE '2024-07-15', 'Corvallis');

SELECT COUNT(*) AS member_count FROM members;
```

output

```
INSERT 0 1

 member_count
--------------
           10
```

Example 2: remove the demonstration row again.

SQL

```sql
DELETE FROM members WHERE email = 'linus@example.org';

SELECT COUNT(*) AS member_count FROM members;
```

output

```
DELETE 1

 member_count
--------------
            9
```

`DELETE 1` reports one removed row, and the count is back to the sample size. Filtering on the unique email rather than the id shows a habit worth keeping: surrogate ids are internal, business keys are what you recognise.

Example 3: a multi-row insert inside a transaction you undo.

SQL

```sql
BEGIN;

INSERT INTO members (full_name, email, joined_on, city) VALUES
    ('Linus Pauling',  'linus@example.org', DATE '2024-07-15', 'Corvallis'),
    ('Barbara McClintock', 'barbara2@example.org', DATE '2024-07-20', 'Cold Spring Harbor');

SELECT COUNT(*) AS member_count FROM members;

ROLLBACK;

SELECT COUNT(*) AS member_count FROM members;
```

output

```
BEGIN
INSERT 0 2

 member_count
--------------
           11

ROLLBACK

 member_count
--------------
            9
```

Two rows arrive in one statement and both vanish with the `ROLLBACK`, so the sample data is exactly what it was.

Example 4: an `UPDATE` you can take back.

SQL

```sql
BEGIN;

UPDATE books SET price = price - 1.00;

SELECT book_id, title, price FROM books ORDER BY book_id LIMIT 5;

ROLLBACK;

SELECT book_id, title, price FROM books ORDER BY book_id LIMIT 5;
```

output

```
BEGIN
UPDATE 11

 book_id |           title            | price
---------+----------------------------+-------
       1 | A Wizard of Earthsea       | 11.99
       2 | The Left Hand of Darkness  | 13.50
       3 | One Hundred Years of Solitude | 14.20
       4 | Love in the Time of Cholera | 12.75
       5 | Pride and Prejudice        |  8.99

ROLLBACK

 book_id |           title            | price
---------+----------------------------+-------
       1 | A Wizard of Earthsea       | 12.99
       2 | The Left Hand of Darkness  | 14.50
       3 | One Hundred Years of Solitude | 15.20
       4 | Love in the Time of Cholera | 13.75
       5 | Pride and Prejudice        |  9.99
```

`UPDATE 11` shows the blast radius: without a `WHERE`, all eleven books were discounted. The `ROLLBACK` puts every price back, which is why practising inside a transaction is the safe habit.

Example 5: a targeted `UPDATE`, including writing NULL back.

SQL

```sql
UPDATE members SET city = 'Boston' WHERE member_id = 5;

SELECT member_id, full_name, city FROM members WHERE member_id = 5;
```

output

```
UPDATE 1

 member_id | full_name     | city
-----------+---------------+--------
         5 | Radia Perlman | Boston
```

Member 5 had no city on record, and now has one. To put the sample data back, assign NULL explicitly:

SQL

```sql
UPDATE members SET city = NULL WHERE member_id = 5;
```

Example 6: a targeted `DELETE`, undone by `ROLLBACK`.

SQL

```sql
BEGIN;

DELETE FROM loans WHERE return_date IS NULL;

SELECT COUNT(*) AS loan_count FROM loans;

ROLLBACK;

SELECT COUNT(*) AS loan_count FROM loans;
```

output

```
BEGIN
DELETE 4

 loan_count
------------
           8

ROLLBACK

 loan_count
------------
          12
```

The four open loans disappear, the count falls to 8, and the rollback brings them back. `DELETE` deletes rows; to remove the table itself you would need `DROP TABLE loans;`.

Example 7: copy rows into a new table.

SQL

```sql
CREATE TABLE loan_archive (LIKE loans);

INSERT INTO loan_archive
SELECT * FROM loans WHERE return_date IS NOT NULL;

SELECT COUNT(*) AS archived_loans FROM loan_archive;

DROP TABLE loan_archive;
```

output

```
CREATE TABLE
INSERT 0 8

 archived_loans
----------------
              8

DROP TABLE
```

The `LIKE` clause builds an empty table with loans' columns, the `INSERT ... SELECT` fills it with the eight returned loans, and the final `DROP TABLE` tidies up so the sample schema stays as this book describes it.

Example 8: parameters, the safe way to accept a value.

SQL

```sql
PREPARE member_by_email (varchar) AS
    SELECT member_id, full_name, city
    FROM members
    WHERE email = $1;

EXECUTE member_by_email ('grace@example.org');
```

output

```
PREPARE
 member_id | full_name   |   city
-----------+-------------+----------
         3 | Grace Hopper | Arlington
```

`$1` is a placeholder. The server parses the statement once and treats every executed value as data, so an address such as `' OR '1'='1` is just a strange string and not a change to the query. Application drivers expose the same idea under names like parameter binding or a prepared statement.

6.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`UPDATE` or `DELETE` without `WHERE`	every row changes, sometimes before you notice	write the `WHERE` first, test it with `SELECT`, then change the verb
Gluing user input into the SQL text	invites SQL injection	parameters (`$1`, ...) or a driver's binding
`BEGIN` without `COMMIT`	the work is rolled back when the session ends	commit deliberately, or roll back on purpose
Emptying a big table with `DELETE`	deletes row by row and is slow	`TRUNCATE TABLE t;` when you really want it empty
`DROP TABLE` when you meant `DELETE`	the table and its data are gone	double-check the verb, keep backups
Retyping the sample rows after a mistake	ids drift and results stop matching	practice inside `BEGIN ... ROLLBACK`

- Run the `SELECT` version of a change before you run the change itself.
- After any `INSERT`, check with `SELECT COUNT(*)` that the row count moved the way you expected.
- `DELETE` never reuses identity values: a deleted id stays retired.

6.5 Practice exercises

1. Add a member with your own name, then remove that member again.
2. Raise every Science fiction price by 10 percent, look at the result, and then restore the original prices without knowing them in advance.
3. Delete loan 12 and then put the same loan back exactly as it was.
4. Copy every loan from 2025 into a table named `loan_archive_2025`, then drop that table.
5. In one sentence, why is pasting user input into a query dangerous?

Solutions

1. `sql INSERT INTO members (full_name, email, joined_on, city) VALUES ('Your Name', 'you@example.org', DATE '2024-08-01', 'Your City');

DELETE FROM members WHERE email = 'you@example.org';

```
   The email is the safest handle because it is unique.
2. ```sql
   BEGIN;

   UPDATE books SET price = ROUND(price * 1.10, 2)
   WHERE genre = 'Science fiction';

   SELECT title, price FROM books ORDER BY price DESC LIMIT 3;

   ROLLBACK;
```

The rollback restores the stored prices exactly, which a manual "subtract 10 percent" could not do reliably.

3. `sql DELETE FROM loans WHERE loan_id = 12;

INSERT INTO loans (book_id, member_id, loan_date, return_date) VALUES (1, 6, DATE '2025-04-15', NULL);

```
   The replacement row is not loan 12: the identity column hands out the next
   free number, 13, because ids are never reused.
4. ```sql
   CREATE TABLE loan_archive_2025 (LIKE loans);

   INSERT INTO loan_archive_2025
   SELECT * FROM loans
   WHERE loan_date >= DATE '2025-01-01' AND loan_date < DATE '2026-01-01';

   DROP TABLE loan_archive_2025;
```

All twelve sample loans are dated in 2025, so the table holds twelve rows.

5. Because the input can close the string it sits in and add its own SQL, which lets an attacker read, change or destroy data they should never reach.

6.6 Mini test

1. Multiple choice: which clause keeps an `UPDATE` from touching every row? - a) `LIMIT` - b) `WHERE` - c) `ORDER BY` - d) `SET`
2. True or false: after `BEGIN`, changes are permanent the moment you press Enter on the statement.
3. Multiple choice: which statement removes rows but keeps the table? - a) `DROP TABLE` - b) `DELETE` - c) `TRUNCATE DATABASE` - d) `ALTER TABLE`
4. Short code question: write an `INSERT` that adds the author "Test Author" from "Nowhere", leaving `birth_date` empty.
5. True or false: `INSERT INTO t SELECT * FROM s;` copies rows produced by a query.
6. Multiple choice: after you delete a row and insert a similar one, the new row's identity value is: - a) the same number as the deleted row - b) the next number that was never used - c) chosen at random - d) always zero
7. Short answer: what is SQL injection, in one sentence?

Answer key and explanations

1. b - `WHERE` names the rows to change; without it the update is global.
2. False - the work waits for `COMMIT`, and `ROLLBACK` (or closing the session) discards it.
3. b - `DELETE` removes rows; `DROP TABLE` removes the table itself.
4. `sql INSERT INTO authors (first_name, last_name, birth_date, country) VALUES ('Test', 'Author', NULL, 'Nowhere');` The omitted id is filled by the identity column and `NULL` records no birth date.
5. True - the query's result rows are appended to the target table.
6. b - identity values are never reused, which is why "the next free number" appears.
7. It is an attack where user-supplied text changes the meaning of the SQL statement, defeated by sending values as parameters instead of concatenating them.

Chapter 7 - NULL, dates and conditional expressions

7.1 Summary

- NULL means "no value recorded", which is not the same as zero or an empty string.
- A comparison with NULL evaluates to unknown, and `WHERE` discards rows whose condition is unknown.
- `IS NULL` and `IS NOT NULL` are the only safe tests for NULL.
- `COALESCE` returns its first argument that is not NULL, and `NULLIF` returns NULL when its two arguments are equal.
- `COUNT(column)` ignores NULLs while `COUNT(*)` counts every row.
- `DATE` holds a calendar day, `TIME` a time of day, `TIMESTAMP` both, and `INTERVAL` a span of time.
- Date arithmetic uses intervals: a date plus an interval is a timestamp, and a timestamp minus a timestamp is an interval.
- `EXTRACT` pulls one field, such as the year or the month, out of a date or timestamp.
- `CASE WHEN` returns the first matching branch and may be written in its simple or searched form.
- `CASE` can appear wherever a value is expected, including `SELECT`, `WHERE`, `ORDER BY` and `UPDATE`.

7.2 Concepts

Three-valued logic

output

```
condition        result      does the row pass WHERE?
-------------------------------------------------------
price > 13       true/false  yes when true
NULL > 13        unknown     no, unknown is not true
NOT unknown      unknown     no
```

"Unknown" behaves like a third truth value, so `NOT (NULL > 13)` is also unknown. That single rule explains most surprising query results involving NULL.

The NULL-handling functions

Function	Returns	Example
`IS NULL`	true when the value is NULL	`city IS NULL`
`COALESCE(a, b, ...)`	the first argument that is not NULL	`COALESCE(city, 'Unknown')`
`NULLIF(a, b)`	NULL when `a = b`, otherwise `a`	`NULLIF(genre, 'Fiction')`

`COALESCE` is how you fill report columns; `NULLIF` is how you turn a sentinel value such as `0` or `'n/a'` back into a real NULL.

Dates, times and intervals

Type	Holds	Example literal
`DATE`	a calendar day	`DATE '2025-01-05'`
`TIME`	a time of day	`TIME '09:30:00'`
`TIMESTAMP`	a day and a time	`TIMESTAMP '2025-01-05 09:30:00'`
`INTERVAL`	a span of time	`INTERVAL '14 days'`

Useful operations, all valid in PostgreSQL:

SQL

```sql
loan_date + INTERVAL '14 days'          -- a date plus an interval is a timestamp
return_date - loan_date                 -- a timestamp minus a timestamp is an interval in days
EXTRACT(YEAR FROM birth_date)           -- one field, as a number
AGE(return_date, loan_date)             -- the interval between two dates
CURRENT_DATE                            -- today, changes as the calendar does
```

CASE expressions

SQL

```sql
-- searched form: each WHEN holds a full condition
CASE WHEN price < 12 THEN 'budget' ELSE 'standard' END

-- simple form: one value is compared against a list
CASE genre WHEN 'Romance' THEN 'love stories' ELSE genre END
```

The branches are tested in order and the first match wins. Without an `ELSE`, a `CASE` that matches nothing returns NULL, so reports usually end with `ELSE`.

7.3 Examples

Example 1: a NULL value silently drops out.

SQL

```sql
SELECT first_name, last_name
FROM authors
WHERE birth_date > DATE '1950-01-01';
```

output

```
 first_name |  last_name
------------+-------------
 Chimamanda | Ngozi Adichie
 Kazuo      | Ishiguro
```

Only two authors were born after 1950. Frank Herbert is not among them, and not because he was born earlier: his birth date is NULL, so the comparison is unknown and the row is filtered out.

Example 2: include the unknown on purpose.

SQL

```sql
SELECT first_name, last_name
FROM authors
WHERE birth_date > DATE '1950-01-01' OR birth_date IS NULL;
```

output

```
 first_name |  last_name
------------+-------------
 Chimamanda | Ngozi Adichie
 Kazuo      | Ishiguro
 Frank      | Herbert
```

Adding `OR birth_date IS NULL` catches the unknown case as well.

Example 3: `COALESCE` fills a report column.

SQL

```sql
SELECT member_id, full_name, COALESCE(city, 'Unknown') AS city
FROM members;
```

output

```
 member_id |     full_name     |   city
-----------+-------------------+-----------
         1 | Ada Lovelace      | London
         2 | Alan Turing       | Manchester
         3 | Grace Hopper      | Arlington
         4 | Katherine Johnson | Hampton
         5 | Radia Perlman     | Unknown
         6 | Margaret Hamilton | Boston
         7 | Barbara Liskov    | Boston
         8 | Shafi Goldwasser  | New York
         9 | Tim Berners-Lee   | London
```

Member 5 has no city in the table; the report still shows a readable word.

Example 4: `NULLIF` turns a value into NULL.

SQL

```sql
SELECT book_id, title, genre, NULLIF(genre, 'Fiction') AS not_fiction
FROM books
WHERE book_id IN (6, 7, 3);
```

output

```
 book_id |      title      |  genre  | not_fiction
---------+-----------------+---------+-------------
       6 | Norwegian Wood  | Fiction |
       7 | Kafka on the Shore | Fiction |
       3 | One Hundred Years of Solitude | Magical realism | Magical realism
```

For the two Fiction books `NULLIF` returns NULL, and for book 3 it returns the genre unchanged. This is the trick used to convert placeholder values into true NULLs before aggregation.

Example 5: date arithmetic on the loans.

SQL

```sql
SELECT loan_id,
       loan_date,
       loan_date + INTERVAL '14 days' AS due_date,
       return_date - loan_date AS days_kept
FROM loans
WHERE loan_id IN (1, 2, 6, 9);
```

output

```
 loan_id | loan_date  |      due_date      | days_kept
---------+------------+--------------------+-----------
       1 | 2025-01-05 | 2025-01-19 00:00:00 |         14
       2 | 2025-01-05 | 2025-01-19 00:00:00 |         21
       6 | 2025-02-18 | 2025-03-04 00:00:00 |         14
       9 | 2025-03-14 | 2025-04-01 00:00:00 |         18
```

Adding an interval to a date produces a timestamp, which psql prints with a `00:00:00` time part. Subtracting two dates yields a whole number of days: loan 2 was out for 21 days even though its due date was the same as loan 1's.

Example 6: `EXTRACT` pulls a field out.

SQL

```sql
SELECT first_name, last_name,
       CAST(EXTRACT(YEAR FROM birth_date) AS INTEGER) AS born
FROM authors
WHERE birth_date IS NOT NULL;
```

output

```
 first_name |  last_name  | born
------------+-------------+------
 Ursula     | Le Guin     | 1929
 Gabriel    | Garcia Marquez | 1927
 Jane       | Austen      | 1775
 Haruki     | Murakami    | 1949
 Chimamanda | Ngozi Adichie | 1977
 Kazuo      | Ishiguro    | 1954
```

`EXTRACT` returns a numeric value in PostgreSQL, and the `CAST` turns it into a plain integer for the report.

Example 7: `AGE` measures a span.

SQL

```sql
SELECT loan_id, AGE(return_date, loan_date) AS out_for
FROM loans
WHERE loan_id IN (2, 9);
```

output

```
 loan_id | out_for
---------+----------
       2 | 21 days
       9 | 18 days
```

`AGE` returns an interval, printed here as a number of days.

Example 8: a searched `CASE` labels the prices.

SQL

```sql
SELECT title, price,
       CASE
           WHEN price < 12 THEN 'budget'
           WHEN price < 14 THEN 'standard'
           ELSE 'premium'
       END AS price_band
FROM books
ORDER BY price;
```

output

```
           title            | price | price_band
----------------------------+-------+------------
 Pride and Prejudice        |  9.99 | budget
 The Word for World is Forest | 10.99 | budget
 Norwegian Wood             | 11.40 | budget
 Half of a Yellow Sun       | 12.50 | standard
 A Wizard of Earthsea       | 12.99 | standard
 Kafka on the Shore         | 13.00 | standard
 The Dispossessed           | 13.25 | standard
 Love in the Time of Cholera | 13.75 | standard
 The Left Hand of Darkness  | 14.50 | premium
 Dune                       | 14.99 | premium
 One Hundred Years of Solitude | 15.20 | premium
```

Three prices land in each band. The conditions are tested in order, so a price of 11.40 never reaches the second branch.

Example 9: the simple form of `CASE`.

SQL

```sql
SELECT book_id, genre,
       CASE genre
           WHEN 'Romance' THEN 'love stories'
           WHEN 'Fiction' THEN 'general fiction'
           ELSE genre
       END AS friendly_genre
FROM books
WHERE book_id IN (4, 6, 9);
```

output

```
 book_id |  genre  |  friendly_genre
---------+---------+-----------------
       4 | Romance | love stories
       6 | Fiction | general fiction
       9 | Science fiction | Science fiction
```

The simple form compares one value against each `WHEN`, and the `ELSE` keeps the original label for anything unmatched.

7.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Treating NULL as zero or `''`	unknown is not a value, so arithmetic and comparisons mislead	decide per column whether NULL or a default is right
`col = NULL`	the condition is unknown and matches nothing	`IS NULL`
`CASE` without `ELSE` that should have one	unmatched rows return NULL in a report	end with `ELSE` when every row needs a label
Dates stored as text	sorting and arithmetic fail or guess the format	`DATE`, `TIMESTAMP`
`date + 7` with mixed types	works for `DATE` but fails for `TIMESTAMP`	`+ INTERVAL '7 days'` everywhere
`CURRENT_DATE` in a report you compare later	the value moves with the calendar	pass a fixed date when the answer must be stable

- `COALESCE` is for display; fix the missing data if the column matters.
- Store instants in `TIMESTAMP`, whole days in `DATE`, never text.
- `CASE` reads best when every branch returns the same type.

7.5 Practice exercises

1. Show every member with the city replaced by `Unknown` where none is recorded.
2. How many days was loan 9 kept, and what due date would a 14-day rule give it?
3. Label every book `budget` when the price is below 12 and `premium` otherwise, using `CASE`.

4. What do `COALESCE(NULL, NULL, 'x')` and `NULLIF(5, 5)` return?
5. Why is `WHERE joined_on > '2024-01-01'` risky when the column is `TEXT`?

Solutions

1. `sql SELECT member_id, full_name, COALESCE(city, 'Unknown') AS city FROM members;`
2. `sql SELECT loan_date + INTERVAL '14 days' AS due_date, return_date - loan_date AS days_kept FROM loans WHERE loan_id = 9;` The due date is 2025-03-28 00:00:00 and the book was kept for 18 days.
3. `sql SELECT title, CASE WHEN price < 12 THEN 'budget' ELSE 'premium' END AS band FROM books ORDER BY price;` Three books are budget and eight are premium.
4. `'x'`, because the first two arguments are NULL and the first non-NULL value wins; and NULL, because the two arguments are equal.
5. Text comparison follows character order, so the filter depends on the stored format and on collation, and a day written as `01/02/2024` sorts next to `01/03/2024` by luck rather than by time. A real `DATE` column compares as days.

7.6 Mini test

1. Multiple choice: what does NULL represent? - a) the number zero - b) an empty string - c) no recorded value - d) the value false
2. True or false: `WHERE x = NULL` returns the rows where `x` is NULL.
3. Multiple choice: which function returns its first argument that is not NULL? - a) `NULLIF` - b) `COALESCE` - c) `ISNULL` - d) `IFNULL`
4. Short code question: write a query showing each member with `Unknown` in place of a missing city.
5. True or false: `loan_date + INTERVAL '7 days'` is valid PostgreSQL.
6. Multiple choice: what does a `CASE` expression return when no `WHEN` matches and there is no `ELSE`? - a) an error - b) NULL - c) zero - d) the first `WHEN` value
7. Short answer: which function returns NULL when its two arguments are equal?

Answer key and explanations

1. c - NULL records the absence of a value, not a value itself.
2. False - the comparison is unknown, so no row qualifies; `IS NULL` is the correct test.
3. b - `COALESCE` walks its arguments and returns the first non-NULL one.
4. `sql SELECT full_name, COALESCE(city, 'Unknown') AS city FROM members;`
5. True - dates and intervals combine, producing a timestamp.
6. b - no branch matched and no fallback was given, so the result is NULL.
7. `NULLIF(a, b)`, which returns NULL when `a` and `b` are equal.

Chapter 8 - Joins

8.1 Summary

- A join combines rows from two tables by matching values between their columns.
- The join condition normally pairs the foreign key of one table with the primary key of the other.
- An `INNER JOIN` keeps only the rows that match on both sides.
- A `LEFT JOIN` keeps every row of the left table and fills the right side with NULLs where nothing matches.
- A `RIGHT JOIN` is the mirror image: every row of the right table survives.
- A `FULL OUTER JOIN` keeps every row of both tables, matching where it can.
- A `SELF JOIN` joins a table to itself and needs two aliases to tell the copies apart.
- A `CROSS JOIN` pairs every row of one table with every row of the other.
- When a join multiplies rows, the filters and counts that follow apply to the multiplied result.

8.2 Concepts

What a join does

output

```
authors                          books
author_id first_name             book_id title           author_id
     1    Ursula      \              1    A Wizard...          1
     2    Gabriel      >-----<       2     The Left...          1
     3    Jane                \     3     One Hundred...       2
```

The `ON` clause states which columns must agree. A row of the left table is paired with every row of the right table that satisfies the condition, so a one-to-many relationship produces as many result rows as there are children.

The join types

Join	Keeps	Typical use
`JOIN` (INNER)	only rows matching on both sides	books with their authors
`LEFT JOIN`	all left rows, NULLs on the right where no match	authors including those with no books
`RIGHT JOIN`	all right rows, NULLs on the left	books including never-borrowed ones
`FULL OUTER JOIN`	all rows of both tables	reconciliation between two tables
`CROSS JOIN`	every left row paired with every right row	generating combinations

`OUTER` is optional in PostgreSQL, so `LEFT OUTER JOIN` and `LEFT JOIN` are the same statement.

ON and WHERE in an outer join

SQL

```
FROM members AS m
LEFT JOIN loans AS l
    ON l.member_id = m.member_id AND l.return_date IS NULL
```

The condition inside `ON` decides which right rows are considered a match. A condition on the right table written in `WHERE` instead filters after the join, which discards the NULL rows the outer join had produced and quietly turns it into an inner join.

A table can join itself

SQL

```
FROM authors AS a1
JOIN authors AS a2 ON a1.country = a2.country
                  AND a1.author_id < a2.author_id
```

Two aliases stand for two logical copies of the same table. The inequality on the ids keeps each pair once instead of twice, and keeps a row from pairing with itself.

8.3 Examples

Example 1: an inner join between books and authors.

SQL

```
SELECT b.book_id, b.title, a.first_name, a.last_name
FROM books AS b
JOIN authors AS a ON a.author_id = b.author_id
ORDER BY b.book_id
LIMIT 6;
```

output

```
 book_id |           title            | first_name |  last_name
---------+----------------------------+------------+-------------
       1 | A Wizard of Earthsea       | Ursula     | Le Guin
       2 | The Left Hand of Darkness  | Ursula     | Le Guin
       3 | One Hundred Years of Solitude | Gabriel   | Garcia Marquez
       4 | Love in the Time of Cholera | Gabriel    | Garcia Marquez
       5 | Pride and Prejudice        | Jane       | Austen
       6 | Norwegian Wood             | Haruki     | Murakami
```

Six of eleven rows; every book has an author, so the full result has all eleven books. `JOIN` without a keyword means `INNER JOIN`.

Example 2: a left join that finds authors with no books.

SQL

```
SELECT a.author_id, a.first_name, a.last_name, b.book_id
FROM authors AS a
LEFT JOIN books AS b ON b.author_id = a.author_id
WHERE b.book_id IS NULL;
```

output

```
 author_id | first_name | last_name | book_id
-----------+------------+-----------+---------
         7 | Kazuo      | Ishiguro  |
```

Author 7 wrote none of the eleven books, so the right side is NULL. Without the `WHERE`, the same left join would return twelve rows: eleven with a book and one with NULLs for author 7.

Example 3: a left join that finds a member with no loans.

SQL

```sql
SELECT m.member_id, m.full_name, l.loan_id
FROM members AS m
LEFT JOIN loans AS l ON l.member_id = m.member_id
WHERE l.loan_id IS NULL;
```

output

```
 member_id |   full_name   | loan_id
-----------+---------------+---------
         9 | Tim Berners-Lee |
```

Tim Berners-Lee joined the library but never borrowed anything.

Example 4: a right join that finds a book that was never borrowed.

SQL

```sql
SELECT l.loan_id, b.book_id, b.title
FROM loans AS l
RIGHT JOIN books AS b ON b.book_id = l.book_id
WHERE l.loan_id IS NULL;
```

output

```
 loan_id | book_id |           title
---------+---------+------------------------
         |      11 | The Word for World is Forest
```

Reading the join from the right, every book is kept even when no loan matched, and the filter isolates the one book with no loan history.

Example 5: a full outer join over authors and books.

SQL

```sql
SELECT a.author_id AS author_side, b.book_id AS book_side
FROM authors AS a
FULL OUTER JOIN books AS b ON b.author_id = a.author_id
WHERE a.author_id IS NULL OR b.book_id IS NULL;
```

output

```
 author_side | book_side
-------------+-----------
           7 |
```

The full join would return twelve rows, eleven matched and one with the author of no books. The `WHERE` keeps only the unmatched rows from either side, and here no book lacks an author,

so one row comes back.

Example 6: a self join finds authors from the same country.

SQL

```sql
SELECT a1.first_name || ' ' || a1.last_name AS author_one,
       a2.first_name || ' ' || a2.last_name AS author_two,
       a1.country
FROM authors AS a1
JOIN authors AS a2
    ON a1.country = a2.country
   AND a1.author_id < a2.author_id
ORDER BY a1.author_id;
```

output

```
       author_one      |    author_two    |    country
-----------------------+------------------+---------------
 Ursula Le Guin        | Frank Herbert    | United States
 Haruki Murakami       | Kazuo Ishiguro   | Japan
```

Two pairs share a country. The strict inequality makes each pair appear once, in one direction only.

Example 7: a cross join counts every combination.

SQL

```sql
SELECT COUNT(*) AS author_book_pairs
FROM authors
CROSS JOIN books;
```

output

```
 author_book_pairs
-------------------
                77
```

Seven authors times eleven books is 77, which is what a forgotten `ON` produces by accident.

Example 8: `ON` versus `WHERE` in a left join.

SQL

```sql
SELECT m.member_id, m.full_name, l.loan_id
FROM members AS m
LEFT JOIN loans AS l
    ON l.member_id = m.member_id AND l.return_date IS NULL
ORDER BY m.member_id;
```

output

```
 member_id |     full_name     | loan_id
-----------+-------------------+---------
         1 | Ada Lovelace      |
         2 | Alan Turing       |
         3 | Grace Hopper      |       5
         4 | Katherine Johnson |
         5 | Radia Perlman     |       7
         6 | Margaret Hamilton |      12
         7 | Barbara Liskov    |
         8 | Shafi Goldwasser  |      10
         9 | Tim Berners-Lee   |
```

The condition on `return_date` sits in `ON`, so all nine members appear and each shows the open loan, if any. Moving that same condition into `WHERE` would drop every member without an open loan, leaving only four rows.

8.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Joining without `ON`	produces the cross product, a huge wrong result	always state the matching columns
Filtering the right table in `WHERE` after a `LEFT JOIN`	removes the NULL rows and turns the join into an inner join	put right-table conditions in `ON`
Unqualified column names when two tables share one	"column reference is ambiguous" error	alias both tables and prefix every column
Using `DISTINCT` to hide rows a one-to-many join multiplied	hides the real cardinality question	decide whether you want one row per child or an aggregate
`UNION` to emulate a `FULL OUTER JOIN`	slower and harder to read	use `FULL OUTER JOIN`

- Read a join aloud: "every book with the author whose id matches".
- When a count looks too high after a join, suspect a one-to-many step and group instead of deduplicating.
- `LEFT JOIN` plus `IS NULL` is the standard idiom for "rows with no children".

8.5 Practice exercises

1. List every book title together with the author's full name.
2. Find the authors who have written no books.
3. Find the books that have never been borrowed.
4. List each pair of members who live in the same city.
5. Why does joining `books` with `loans` return more than eleven rows?

Solutions

1. `sql SELECT b.title, a.first_name || ' ' || a.last_name AS author FROM books AS b JOIN authors AS a ON a.author_id = b.author_id ORDER BY b.book_id;`
2. `sql SELECT a.first_name, a.last_name FROM authors AS a LEFT JOIN books AS b ON b.author_id = a.author_id WHERE b.book_id IS NULL;` One row: Kazuo Ishiguro.
3. `sql SELECT b.title FROM books AS b LEFT JOIN loans AS l ON l.book_id = b.book_id WHERE l.loan_id IS NULL;` One row: The Word for World is Forest.
4. `sql SELECT m1.full_name, m2.full_name, m1.city FROM members AS m1 JOIN members AS m2 ON m1.city = m2.city AND m1.member_id < m2.member_id;` Two pairs: Ada Lovelace and Tim Berners-Lee in London, Margaret Hamilton and Barbara Liskov in Boston.
5. Because the join returns one row per loan, and two books were borrowed twice each, so eleven books produce twelve rows.

8.6 Mini test

1. Multiple choice: which join keeps only the rows that match on both sides? - a) `LEFT JOIN` - b) `INNER JOIN` - c) `FULL OUTER JOIN` - d) `CROSS JOIN`
2. True or false: a `LEFT JOIN` can put NULLs from the right table into the result.
3. Multiple choice: which join pairs every row of one table with every row of the other? - a) `SELF JOIN` - b) `CROSS JOIN` - c) `FULL OUTER JOIN` - d) `RIGHT JOIN`
4. Short code question: write a query returning each book title with the author's last name.
5. True or false: joining two tables without an `ON` clause raises an error in PostgreSQL.
6. Multiple choice: in a `LEFT JOIN`, a filter on the right table written in `WHERE` does what? - a) filters the right table before joining - b) removes the NULL rows, effectively making it an inner join - c) nothing at all - d) raises an error
7. Short answer: how do you find the rows of the left table that have no match on the right?

Answer key and explanations

1. b - an inner join is the intersection of both sides.
2. True - that is the point of an outer join: unmatched left rows survive with NULLs on the right.
3. b - the cross product has no matching condition.
4. `sql SELECT b.title, a.last_name FROM books AS b JOIN authors AS a ON a.author_id = b.author_id;`
5. False - it is legal and produces the cross product, which is why the count explodes when the `ON` clause is forgotten.
6. b - the NULL-filled rows can never satisfy the filter, so they disappear.
7. Add `WHERE right_table.key IS NULL` after a `LEFT JOIN`.

Chapter 9 - Aggregating, grouping and combining results

9.1 Summary

- Aggregate functions fold many rows into one value: `COUNT`, `SUM`, `AVG`, `MIN` and `MAX`.
- `COUNT(*)` counts rows, `COUNT(column)` counts non-NULL values, and `COUNT(DISTINCT column)` counts unique ones.
- `GROUP BY` buckets rows and produces one result row per bucket.
- Every selected column must either sit inside an aggregate or appear in the `GROUP BY`.
- `WHERE` filters rows before grouping, while `HAVING` filters the buckets afterwards.
- `UNION` combines result sets and removes duplicates, while `UNION ALL` keeps every row.
- `INTERSECT` keeps the rows both queries return, and `EXCEPT` keeps only the rows unique to the first.
- A subquery is a query nested inside another one and may return a single value, a single column, or a table.
- `EXISTS` tests whether a subquery returns any row at all, and is the idiomatic way to ask "is there a match".
- A view is a named query stored in the database, so you can select from it like a table.

9.2 Concepts

The aggregate functions

Function	Computes	With NULLs
`COUNT(*)`	number of rows	counts every row
`COUNT(col)`	number of values	ignores NULLs
`SUM(col)`	total	ignores NULLs
`AVG(col)`	mean of the values	ignores NULLs, so it averages only what is there
`MIN(col)` / `MAX(col)`	smallest / largest value	ignores NULLs

`AVG` on `NUMERIC` can print a long tail of decimals, so reports usually wrap it in `ROUND(avg, 2)`.

The grouping pipeline

output

```
    all rows of the table
         |
    WHERE condition          -> rows that survive the filter
         |
    GROUP BY bucket key      -> one bucket per distinct value
         |
    aggregate per bucket     -> one result row per bucket
         |
    HAVING bucket condition  -> buckets that survive
         |
    ORDER BY, LIMIT
```

A query without `GROUP BY` is a grouping of one: the whole table is a single bucket, which is why `SELECT COUNT(*) FROM books;` returns exactly one row.

Set operations

Operator	Keeps
`UNION`	rows from both queries, duplicates removed
`UNION ALL`	rows from both queries, duplicates kept
`INTERSECT`	rows returned by both queries
`EXCEPT`	rows of the first query that the second does not return

Both queries must select the same number of columns with compatible types, and a single `ORDER BY` at the end sorts the combined result.

Subqueries

Position	Shape	Example
In `WHERE` after `=`	one value	`price = (SELECT MAX(price) FROM books)`
After `IN` / `NOT IN`	one column, many rows	`author_id IN (SELECT author_id FROM authors WHERE country = 'Japan')`
After `EXISTS`	any row set	`EXISTS (SELECT 1 FROM loans WHERE ...)`
After `ANY` / `ALL`	one column	`price > ALL (SELECT price FROM books WHERE genre = 'Romance')`
In `FROM`	a table	`FROM (SELECT ...) AS derived`

A subquery that mentions the outer query's columns is correlated: it is evaluated once per outer row, which is exactly what `EXISTS` wants.

Views

SQL

```sql
CREATE VIEW name AS <any query>;
```

A view stores a `SELECT` under a name. Every use runs the stored query, so the view never goes stale, and `DROP VIEW name;` removes it.

9.3 Examples

Example 1: aggregates over the whole table.

SQL

```
SELECT COUNT(*)           AS book_count,
       MIN(price)         AS cheapest,
       MAX(price)         AS dearest,
       ROUND(AVG(price), 2) AS average_price,
       SUM(price)         AS total_price
FROM books;
```

output

```
 book_count | cheapest | dearest | average_price | total_price
------------+----------+---------+---------------+-------------
          11 |     9.99 |   15.20 |         12.96 |      142.56
```

Without `GROUP BY` the eleven books form one bucket, so one row comes back.

Example 2: `COUNT(*)` against `COUNT(column)`.

SQL

```
SELECT COUNT(*)          AS authors,
       COUNT(birth_date) AS authors_with_birth_date
FROM authors;
```

output

```
 authors | authors_with_birth_date
---------+-------------------------
       7 |                       6
```

`COUNT(*)` sees every row; `COUNT(birth_date)` skips the NULL, so the difference of one is Frank Herbert.

Example 3: one bucket per author.

SQL

```
SELECT author_id, COUNT(*) AS books_written
FROM books
GROUP BY author_id
ORDER BY author_id;
```

output

```
 author_id | books_written
-----------+---------------
         1 |             4
         2 |             2
         3 |             1
         4 |             2
         5 |             1
         6 |             1
```

Six rows, one per author who has at least one book. Author 7 has no bucket at all, because grouping starts from the books table and author 7 owns none.

Example 4: grouping a left join keeps the empty author.

SQL

```sql
SELECT a.first_name, a.last_name, COUNT(b.book_id) AS books_written
FROM authors AS a
LEFT JOIN books AS b ON b.author_id = a.author_id
GROUP BY a.author_id, a.first_name, a.last_name
ORDER BY books_written DESC, a.author_id;
```

output

```
 first_name |  last_name  | books_written
------------+-------------+---------------
 Ursula     | Le Guin     |             4
 Gabriel    | Garcia Marquez |           2
 Haruki     | Murakami    |             2
 Jane       | Austen      |             1
 Chimamanda | Ngozi Adichie |           1
 Frank      | Herbert     |             1
 Kazuo      | Ishiguro    |             0
```

Now author 7 appears with a count of zero, because the outer join supplies a NULL book row and `COUNT(b.book_id)` ignores that NULL while `COUNT(*)` would have counted it as one.

Example 5: `HAVING` filters the buckets.

SQL

```sql
SELECT a.last_name, COUNT(*) AS books_written
FROM authors AS a
JOIN books AS b ON b.author_id = a.author_id
GROUP BY a.author_id, a.last_name
HAVING COUNT(*) > 1
ORDER BY books_written DESC, a.author_id;
```

output

```
  last_name  | books_written
-------------+---------------
 Le Guin     |             4
 Garcia Marquez |           2
 Murakami    |             2
```

`HAVING` keeps the buckets with more than one book. A `WHERE` could not do this job, because it sees individual rows rather than the count of a bucket.

Example 6: several aggregates per bucket.

SQL

```sql
SELECT genre, COUNT(*) AS books, ROUND(AVG(price), 2) AS avg_price
FROM books
GROUP BY genre
ORDER BY avg_price DESC;
```

output

```
       genre      | books | avg_price
------------------+-------+-----------
 Magical realism |     1 |     15.20
 Science fiction |     4 |     13.43
 Fantasy         |     1 |     12.99
 Historical fiction |   1 |     12.50
 Fiction         |     2 |     12.20
 Romance         |     2 |     11.87
```

Each genre is one bucket with its own count and average, and the two-decimal rounding keeps the averages readable.

Example 7: `UNION` removes duplicates.

SQL

```sql
SELECT genre FROM books WHERE book_id IN (1, 2)
UNION
SELECT genre FROM books WHERE book_id IN (2, 3)
ORDER BY genre;
```

output

```
       genre
----------------
 Fantasy
 Magical realism
 Science fiction
```

Book 2 is Science fiction on both sides, yet it appears once.

Example 8: `UNION ALL` keeps every row.

SQL

```sql
SELECT genre FROM books WHERE book_id IN (1, 2)
UNION ALL
SELECT genre FROM books WHERE book_id IN (2, 3)
ORDER BY genre;
```

output

```
       genre
----------------
 Fantasy
 Magical realism
 Science fiction
 Science fiction
```

Four rows now, because nothing is deduplicated. `UNION ALL` is the cheaper operator, so use it whenever duplicates cannot occur or do not matter.

Example 9: `INTERSECT` and `EXCEPT`.

SQL

```sql
SELECT author_id FROM books WHERE genre = 'Science fiction'
INTERSECT
SELECT author_id FROM books WHERE published_year > 1970;
```

output

```
 author_id
-----------
          1
```

SQL

```sql
SELECT author_id FROM books WHERE genre = 'Science fiction'
EXCEPT
SELECT author_id FROM books WHERE published_year > 1970;
```

output

```
 author_id
-----------
          6
```

The intersect finds authors with a Science fiction book published after 1970; the except finds the Science fiction authors whose books are all older.

Example 10: a scalar subquery in `WHERE`.

SQL

```sql
SELECT title, price
FROM books
WHERE price = (SELECT MAX(price) FROM books);
```

output

```
            title            | price
-----------------------------+-------
 One Hundred Years of Solitude | 15.20
```

The inner query returns one value, so it can stand where a single value is expected.

Example 11: `IN` with a subquery.

SQL

```sql
SELECT title
FROM books
WHERE author_id IN (SELECT author_id FROM authors WHERE country = 'Japan');
```

output

```
            title
------------------------
 Norwegian Wood
 Kafka on the Shore
```

Two authors are Japanese, and only one of them has books. `IN` is equivalent to comparing the column with each value the subquery returns.

Example 12: `EXISTS` and `NOT EXISTS`.

SQL

```sql
SELECT full_name
FROM members AS m
WHERE EXISTS (SELECT 1 FROM loans AS l WHERE l.member_id = m.member_id)
ORDER BY full_name;
```

output

```
       full_name
----------------------
 Ada Lovelace
 Alan Turing
 Barbara Liskov
 Grace Hopper
 Katherine Johnson
 Margaret Hamilton
 Radia Perlman
 Shafi Goldwasser
```

SQL

```sql
SELECT full_name
FROM members AS m
WHERE NOT EXISTS (SELECT 1 FROM loans AS l WHERE l.member_id = m.member_id);
```

output

```
   full_name
---------------
 Tim Berners-Lee
```

The subquery is correlated: it is re-evaluated for each member, with `m` fixed, and returns a row only when that member has a loan. The selected value `1` is a convention, since `EXISTS` only asks whether a row exists.

Example 13: `ANY` and `ALL`.

SQL

```sql
SELECT title, price
FROM books
WHERE price > ANY (SELECT price FROM books WHERE genre = 'Romance')
ORDER BY price DESC;
```

output

```
           title            | price
----------------------------+-------
 One Hundred Years of Solitude | 15.20
 Dune                       | 14.99
 The Left Hand of Darkness  | 14.50
 Love in the Time of Cholera | 13.75
 The Dispossessed           | 13.25
 Kafka on the Shore         | 13.00
```

`ANY` asks for at least one, so any price above the cheapest Romance book qualifies; the comparison is strict, which is why the cheapest Romance book itself, priced 9.99, is not in the result.

SQL

```sql
SELECT title, price
FROM books
WHERE price > ALL (SELECT price FROM books WHERE genre = 'Romance')
ORDER BY price;
```

output

```
            title            | price
-----------------------------+-------
 The Left Hand of Darkness   | 14.50
 Dune                        | 14.99
 One Hundred Years of Solitude | 15.20
```

`ALL` asks for every one, so only prices above the dearest Romance book qualify.

Example 14: `CASE` inside a grouping query.

SQL

```sql
SELECT CASE
           WHEN price < 12 THEN 'budget'
           WHEN price < 14 THEN 'standard'
           ELSE 'premium'
       END AS price_band,
       COUNT(*) AS books
FROM books
GROUP BY 1
ORDER BY price_band;
```

output

```
 price_band | books
------------+-------
 budget     |     3
 premium    |     3
 standard   |     5
```

The grouping key here is the `CASE` expression itself, referred to by its position in the select list. Repeating the whole expression after `GROUP BY` would be equally correct.

Example 15: a view is a query with a name.

SQL

```sql
CREATE VIEW book_authors AS
SELECT b.title, a.first_name, a.last_name
FROM books AS b
JOIN authors AS a ON a.author_id = b.author_id;

SELECT title, last_name
FROM book_authors
ORDER BY title
LIMIT 3;

DROP VIEW book_authors;
```

output

```
CREATE VIEW
           title            |  last_name
----------------------------+-------------
 A Wizard of Earthsea       | Le Guin
 Dune                       | Herbert
 Half of a Yellow Sun       | Ngozi Adichie
```

The view runs the join every time it is queried, so it always shows the current rows, and dropping it removes nothing but the stored definition.

9.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Selecting a column that is neither grouped nor aggregated	the server rejects the query	group by it, or wrap it in an aggregate
Using `WHERE` to filter an aggregate	`WHERE` runs before grouping, so it cannot see totals	`HAVING COUNT(*) > 1`
`COUNT(col)` when you meant `COUNT(*)`	NULLs are skipped and the number is too low	decide which you want before typing
`UNION` when duplicates are impossible	pays for a sort and a dedup you did not need	`UNION ALL`
Deeply nesting subqueries	unreadable and hard to debug	`WITH` common table expressions, or a view
Assuming `AVG` averages the distinct values	it averages rows, so repeated rows weigh more	deduplicate first if that is the question

- Group, then look at the bucket counts: a bucket of one usually means the key is too specific.
- When two questions need the same subquery, compute it once as a view or a `WITH` query.
- `ROUND` money before printing; exact arithmetic can leave long decimals.

9.5 Practice exercises

1. Show each genre with its number of books and its average price, highest average first.
2. Count the books per author, including the author who has none.
3. List the last names of authors who have written more than one book.
4. List the titles that cost more than the cheapest Romance book.
5. List the members who have never borrowed a book.

Solutions

1. `sql SELECT genre, COUNT(*) AS books, ROUND(AVG(price), 2) AS avg_price FROM books GROUP BY genre ORDER BY avg_price DESC;` Magical realism leads at 15.20, Romance trails at 11.87.

2. `sql SELECT a.last_name, COUNT(b.book_id) AS books_written FROM authors AS a LEFT JOIN books AS b ON b.author_id = a.author_id GROUP BY a.author_id, a.last_name ORDER BY a.author_id;` Kazuo Ishiguro appears with zero. Counting `b.book_id` rather than `*` is what makes the zero appear.
3. `sql SELECT a.last_name FROM authors AS a JOIN books AS b ON b.author_id = a.author_id GROUP BY a.author_id, a.last_name HAVING COUNT(*) > 1;` Le Guin with four books, Garcia Marquez and Murakami with two each.
4. `sql SELECT title FROM books WHERE price > ANY (SELECT price FROM books WHERE genre = 'Romance') ORDER BY price;` Six titles, everything priced above the 9.99 Romance book.
5. `sql SELECT full_name FROM members AS m WHERE NOT EXISTS ( SELECT 1 FROM loans AS l WHERE l.member_id = m.member_id );` One row: Tim Berners-Lee.

9.6 Mini test

1. Multiple choice: which expression ignores NULL values while counting? - a) `COUNT(*)` - b) `COUNT(column)` - c) `SUM(*)` - d) `MAX(*)`
2. True or false: `HAVING` filters the rows before they are grouped.
3. Multiple choice: which set operator removes duplicates? - a) `UNION ALL` - b) `UNION` - c) `INTERSECT` - d) `EXCEPT`
4. Short code question: write a query returning the total price of all books.
5. True or false: a query with `GROUP BY` may select a column that is neither grouped nor aggregated.
6. Multiple choice: what does `EXISTS` test? - a) whether the subquery returns at least one row - b) how many rows the subquery returns - c) whether a column is NULL - d) whether a table exists in the database
7. Short answer: which clause filters groups after the aggregation has run?

Answer key and explanations

1. b - `COUNT(column)` counts non-NULL values; `COUNT(*)` counts every row.
2. False - that is what `WHERE` does; `HAVING` runs after grouping and filters buckets.
3. b - `UNION` deduplicates, while `UNION ALL` keeps every row.
4. `sql SELECT SUM(price) FROM books;` The answer is 142.56.
5. False - every selected column must be grouped or inside an aggregate, or the server reports an error.
6. a - the row values themselves are irrelevant; only the presence of a row matters.
7. `HAVING`, for example `HAVING COUNT(*) > 1`.

Cheat sheet

10.1 A query in order

Clause	Job
`SELECT`	columns and expressions to show
`FROM`	tables the rows come from; joins start here
`WHERE`	row filter, runs before grouping
`GROUP BY`	bucket key for aggregation
`HAVING`	bucket filter, runs after aggregation
`ORDER BY`	sort keys, `ASC`/`DESC`, `NULLS FIRST`/`LAST`
`LIMIT` / `OFFSET`	how many rows to keep, how many to skip

10.2 Operators and predicates

Kind	Operators
Comparison	`=`, `<>`, `<`, `<=`, `>`, `>=`
Logic	`AND`, `OR`, `NOT` (`AND` binds tighter than `OR`)
Ranges and lists	`BETWEEN a AND b` (closed), `IN (...)`, `NOT IN`, `NOT BETWEEN`
Patterns	`LIKE` (case-sensitive), `ILIKE` (case-insensitive), `NOT LIKE`
Wildcards	`%` any run of characters, `_` exactly one
NULL tests	`IS NULL`, `IS NOT NULL` (never `= NULL`)

10.3 Joins

Join	Result
`A JOIN B ON cond`	only matching rows of both
`A LEFT JOIN B ON cond`	all of A, NULLs from B where no match
`A RIGHT JOIN B ON cond`	all of B, NULLs from A where no match
`A FULL OUTER JOIN B ON cond`	all rows of both, matched where possible
`A CROSS JOIN B`	every A row with every B row
Self join	same table twice with two aliases and a `<` tie-breaker

Rules of thumb: put right-table filters in `ON`, not `WHERE`; alias both tables and prefix columns; count a child table with `COUNT(child.key)` after a left join.

10.4 Aggregation and set operations

Function / operator	Notes
`COUNT(*)`	counts rows
`COUNT(col)`	counts non-NULL values
`COUNT(DISTINCT col)`	counts unique values
`SUM`, `AVG`, `MIN`, `MAX`	ignore NULLs; wrap `AVG` in `ROUND(x, 2)`
`UNION`	both queries, duplicates removed
`UNION ALL`	both queries, duplicates kept
`INTERSECT`	rows returned by both
`EXCEPT`	rows of the first not returned by the second
Subquery after `=`, `IN`, `> ANY`, `> ALL`	one value, one column, or any row set
`EXISTS (SELECT 1 ...)`	true when the subquery returns any row

10.5 NULL, dates and CASE

Need	Use
First non-NULL value	`COALESCE(a, b, ...)`
NULL when equal	`NULLIF(a, b)`
Date and time	`DATE`, `TIME`, `TIMESTAMP`, `INTERVAL`
Add or subtract time	`d + INTERVAL '14 days'`, `t2 - t1`
One field	`EXTRACT(YEAR FROM d)` (numeric; cast with `CAST(... AS INTEGER)`)
Span between two dates	`AGE(t2, t1)`
Today	`CURRENT_DATE` (moves with the calendar)
Conditional value	`CASE WHEN c1 THEN v1 WHEN c2 THEN v2 ELSE vN END`

10.6 Changing data

Statement	Shape
Insert	`INSERT INTO t (cols) VALUES (...), (...);`
Insert from a query	`INSERT INTO t (cols) SELECT ... FROM s;`
Update	`UPDATE t SET col = expr WHERE cond;`
Delete	`DELETE FROM t WHERE cond;`
Empty a table fast	`TRUNCATE TABLE t;`
Sandbox a change	`BEGIN; ... ROLLBACK;` then repeat with `COMMIT;`

Statement	Shape
Prepared value	`PREPARE name (type) AS ... WHERE col = $1;` then `EXECUTE name ('value');`

10.7 DDL quick reference

Task	Statement
New table	`CREATE TABLE t (id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, ...);`
New rule	`ALTER TABLE t ADD CONSTRAINT c CHECK (col > 0);`
New column	`ALTER TABLE t ADD COLUMN col type;`
Index for lookups	`CREATE INDEX idx ON t (col);`
Named query	`CREATE VIEW v AS SELECT ...;`
Remove	`DROP TABLE t;` / `DROP VIEW v;`

Constraints: `NOT NULL`, `UNIQUE`, `PRIMARY KEY`, `FOREIGN KEY ... REFERENCES other(col)`, `CHECK (expr)`, `DEFAULT expr`. Types: `INTEGER`, `SMALLINT`, `BIGINT`, `NUMERIC(p,s)` for money, `VARCHAR(n)`, `TEXT`, `DATE`, `TIME`, `TIMESTAMP`, `BOOLEAN`.

10.8 Safe queries checklist

- Send user values as parameters, never glued into the SQL text.
- Write `WHERE` before changing the verb, and test it with `SELECT` first.
- Practise destructive changes inside `BEGIN ... ROLLBACK`.
- Grant each application only the rights it needs.
- Keep the schema in a file and load it with `psql -f`, so the sample data can always be rebuilt.

Final test

1. Multiple choice: which statement creates a table? - a) `MAKE TABLE` - b) `CREATE TABLE` - c) `NEW TABLE` - d) `ADD TABLE`
2. True or false: a column declared `PRIMARY KEY` may contain NULL.
3. Multiple choice: which clause filters rows before grouping? - a) `HAVING` - b) `WHERE` - c) `ORDER BY` - d) `LIMIT`
4. Short code question: write a statement that inserts the author "Test Author" from "Nowhere", leaving the birth date empty.
5. Multiple choice: what does `SELECT DISTINCT` change? - a) the stored table - b) the result set, by removing duplicate rows - c) the column order - d) nothing
6. True or false: `WHERE price = NULL` returns the rows whose price is NULL.

7. Multiple choice: which join keeps every row of the left table? - a) `INNER JOIN` - b) `RIGHT JOIN` - c) `LEFT JOIN` - d) `CROSS JOIN`
8. Short code question: write a query that counts the books.
9. Multiple choice: with `ORDER BY birth_date ASC`, where does PostgreSQL put NULL values? - a) first - b) last - c) it raises an error - d) in a random position
10. True or false: `LIMIT 5` without `ORDER BY` returns a predictable set of rows.
11. Multiple choice: which function returns its first argument that is not NULL? - a) `NULLIF` - b) `COALESCE` - c) `IFNULL` - d) `NVL`
12. Short code question: write a query returning the titles of the books priced above 14.
13. Multiple choice: which clause filters groups after aggregation? - a) `WHERE` - b) `HAVING` - c) `GROUP BY` - d) `LIMIT`
14. True or false: `UNION ALL` keeps duplicate rows.
15. Multiple choice: what does `COUNT(column)` do with NULL values? - a) counts them - b) ignores them - c) turns them into zero - d) raises an error
16. Short code question: write a query showing each member with `Unknown` in place of a missing city.
17. Multiple choice: which set operator keeps only the rows returned by both queries? - a) `UNION` - b) `UNION ALL` - c) `INTERSECT` - d) `EXCEPT`
18. True or false: `UPDATE books SET price = price + 1;` without a `WHERE` changes every book.
19. Multiple choice: in a `LEFT JOIN`, a filter on the right table written in the `WHERE` clause does what? - a) filters the right rows before joining - b) removes the NULL-filled rows, so the join behaves like an inner join - c) changes nothing - d) returns an error
20. Short code question: write a query listing the authors who have written no books.
21. Multiple choice: which data type stores exact amounts of money? - a) `DOUBLE PRECISION` - b) `NUMERIC` - c) `TEXT` - d) `INTEGER`
22. True or false: a `CASE` expression without `ELSE` returns NULL when no `WHEN` matches.
23. Multiple choice: what does `EXISTS` test? - a) how many rows the subquery returns - b) whether the subquery returns at least one row - c) whether a column is NULL - d) whether a table exists
24. Short code question: write a query returning the three most expensive books with their prices.
25. Multiple choice: which technique protects a query from SQL injection? - a) quoting the input with single quotes - b) sending the input as a parameter - c) writing the query on one line - d) using `SELECT *`

11.1 Final test answers

1. b - `CREATE TABLE` defines a new table; the other verbs do not exist.
2. False - a primary key is `NOT NULL` by definition, and uniqueness plus that rule is what identifies a row.
3. b - `WHERE` removes rows before the buckets are formed; `HAVING` works on the buckets afterwards.

4. `sql INSERT INTO authors (first_name, last_name, birth_date, country) VALUES ('Test', 'Author', NULL, 'Nowhere');` The omitted identity id is generated, and NULL records no birth date.
5. b - `DISTINCT` shapes the result set only; the table is untouched.
6. False - the comparison is unknown, so no row qualifies; the correct test is `IS NULL`.
7. c - a left join keeps every left row and fills gaps with NULL.
8. `SELECT COUNT(*) FROM books;` which returns 11.
9. b - PostgreSQL files NULLs last in ascending order and first in descending order unless `NULLS FIRST`/`NULLS LAST` says otherwise.
10. False - without `ORDER BY` the rows have no defined order, so the result is arbitrary.
11. b - `COALESCE` returns the first non-NULL argument.
12. `sql SELECT title FROM books WHERE price > 14;` Three titles: One Hundred Years of Solitude, Dune and The Left Hand of Darkness.
13. b - `HAVING` filters the grouped results, `WHERE` filters the rows.
14. True - `UNION ALL` appends everything and removes nothing, which also makes it the cheaper operator.
15. b - `COUNT(column)` skips NULLs; `COUNT(*)` counts every row.
16. `sql SELECT full_name, COALESCE(city, 'Unknown') AS city FROM members;`
17. c - `INTERSECT` is the overlap; `EXCEPT` is the difference.
18. True - with no `WHERE`, the update applies to all eleven books.
19. b - NULL rows cannot satisfy a filter on the right table, so they are dropped and the outer join quietly becomes an inner join.
20. `sql SELECT a.last_name FROM authors AS a LEFT JOIN books AS b ON b.author_id = a.author_id WHERE b.book_id IS NULL;` One row: Ishiguro.
21. b - `NUMERIC` is exact; floating types round and misreport totals.
22. True - with no matching branch and no `ELSE`, the expression is NULL.
23. b - `EXISTS` only asks whether a row exists; the values do not matter.
24. `sql SELECT title, price FROM books ORDER BY price DESC LIMIT 3;` One Hundred Years of Solitude 15.20, Dune 14.99, The Left Hand of Darkness 14.50.
25. b - parameters travel apart from the statement text, so input can never change what the query means.

Sources

Every URL below was consulted while planning and writing this volume. No text, images or code blocks were copied from these pages; all explanations, examples, exercises, tests and answers were written from scratch, and every SQL statement was written for PostgreSQL and traced by hand against the sample schema.

- https://www.w3schools.com/sql/
- https://www.w3schools.com/sql/sql_intro.asp

- https://www.w3schools.com/sql/sql_syntax.asp
- https://www.w3schools.com/sql/sql_select.asp
- https://www.w3schools.com/sql/sql_distinct.asp
- https://www.w3schools.com/sql/sql_where.asp
- https://www.w3schools.com/sql/sql_orderby.asp
- https://www.w3schools.com/sql/sql_insert.asp
- https://www.w3schools.com/sql/sql_null_values.asp
- https://www.w3schools.com/sql/sql_update.asp
- https://www.w3schools.com/sql/sql_top.asp
- https://www.w3schools.com/sql/sql_aggregate_functions.asp
- https://www.w3schools.com/sql/sql_like.asp
- https://www.w3schools.com/sql/sql_alias.asp
- https://www.w3schools.com/sql/sql_join.asp
- https://www.w3schools.com/sql/sql_union.asp
- https://www.w3schools.com/sql/sql_groupby.asp
- https://www.w3schools.com/sql/sql_exists.asp
- https://www.w3schools.com/sql/sql_select_into.asp
- https://www.w3schools.com/sql/sql_case.asp
- https://www.w3schools.com/sql/sql_isnull.asp
- https://www.w3schools.com/sql/sql_stored_procedures.asp
- https://www.w3schools.com/sql/sql_comments.asp
- https://www.w3schools.com/sql/sql_create_db.asp
- https://www.w3schools.com/sql/sql_create_table.asp
- https://www.w3schools.com/sql/sql_constraints.asp
- https://www.w3schools.com/sql/sql_create_index.asp
- https://www.w3schools.com/sql/sql_dates.asp
- https://www.w3schools.com/sql/sql_view.asp
- https://www.w3schools.com/sql/sql_injection.asp
- https://www.w3schools.com/sql/sql_parameterized_queries.asp
- https://www.w3schools.com/sql/sql_ref_keywords.asp
- https://www.postgresql.org/docs/current/tutorial.html
- https://www.postgresql.org/docs/current/sql-syntax.html
- https://www.postgresql.org/docs/current/sql-select.html
- https://www.postgresql.org/docs/current/sql-insert.html
- https://www.postgresql.org/docs/current/sql-update.html
- https://www.postgresql.org/docs/current/sql-delete.html
- https://www.postgresql.org/docs/current/queries-table-expressions.html
- https://www.postgresql.org/docs/current/queries-union.html

- https://www.postgresql.org/docs/current/functions-aggregate.html
- https://www.postgresql.org/docs/current/functions-conditional.html
- https://www.postgresql.org/docs/current/functions-datetime.html
- https://www.postgresql.org/docs/current/functions-comparison.html
- https://www.postgresql.org/docs/current/functions-matching.html
- https://www.postgresql.org/docs/current/ddl.html
- https://www.postgresql.org/docs/current/ddl-constraints.html
- https://www.postgresql.org/docs/current/sql-createtable.html
- https://www.postgresql.org/docs/current/sql-createindex.html
- https://www.postgresql.org/docs/current/sql-createview.html
- https://www.postgresql.org/docs/current/sql-prepare.html
- https://www.postgresql.org/docs/current/app-psql.html